

Day 5：数据集获取 + 格式转换 — 详细教程

目录

- [一、今日目标与验收标准](#)
 - [二、YOLOv11 原理速通](#)
 - [三、YOLO 标注格式详解](#)
 - [四、常见标注格式对比](#)
 - [五、数据集获取渠道与实操](#)
 - [六、数据标注工具使用教学](#)
 - [七、DataConverter 实现：VOC XML → YOLO TXT](#)
 - [八、DataConverter 实现：COCO JSON → YOLO TXT](#)
 - [九、DataConverter 实现：LabelMe JSON → YOLO TXT](#)
 - [十、数据集目录结构组织](#)
 - [十一、data.yaml 配置文件生成](#)
 - [十二、数据集划分策略](#)
 - [十三、格式转换完整 workflow](#)
 - [十四、各组场景数据集准备指南](#)
 - [十五、常见问题排查](#)
-

一、今日目标与验收标准

1.1 今日任务总览

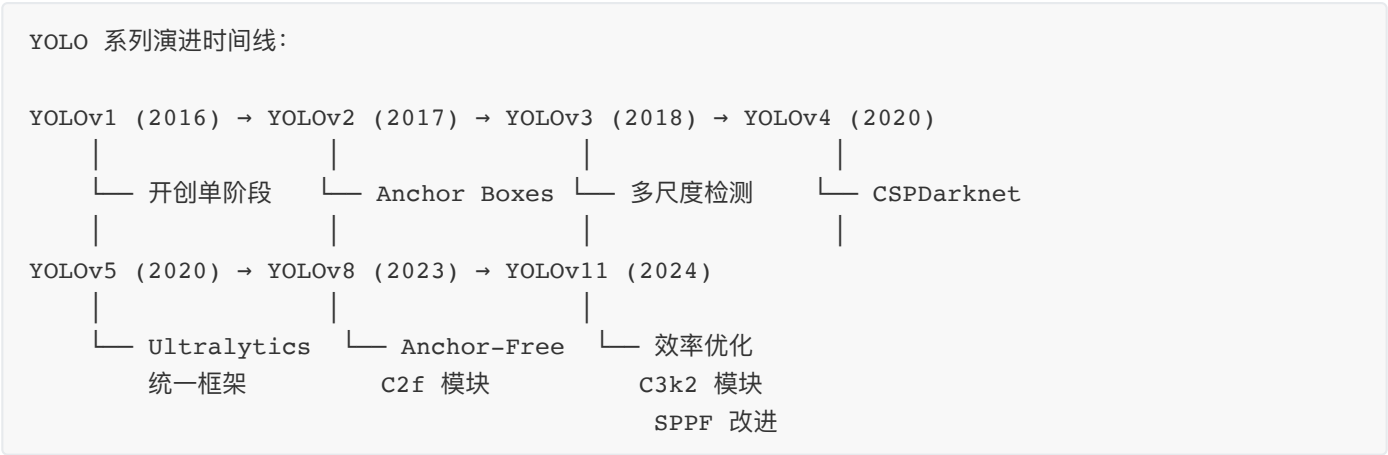
Day 5 是**第二阶段（数据与模型）**的第一天，进入项目核心——目标检测的数据准备工作。高质量的数据集是模型训练效果的决定性因素，而格式转换是将各种来源的数据统一为 YOLO 可用格式的关键环节。

├── val/	# 验证集标注
├── test/	# 测试集标注
└── data.yaml	# 数据集配置文件
├── tools/	# 【新增目录】# 命令行工具（面向开发者）
│ ├── convert_voc.py	# VOC 格式转换脚本
│ ├── convert_coco.py	# COCO 格式转换脚本
│ ├── convert_labelme.py	# LabelMe 格式转换脚本
│ ├── verify_dataset.py	# 数据集验证脚本
│ ├── fix_bbox.py	# YOLO 标注修复（安全保障）
│ └── fix_xml_coords.py	# XML 标注修复（安全保障）
├── models/	# 【新增目录】模型文件
│ └── yolov11n.pt	# 预训练权重（后续 Day 下载）
└── docs/	

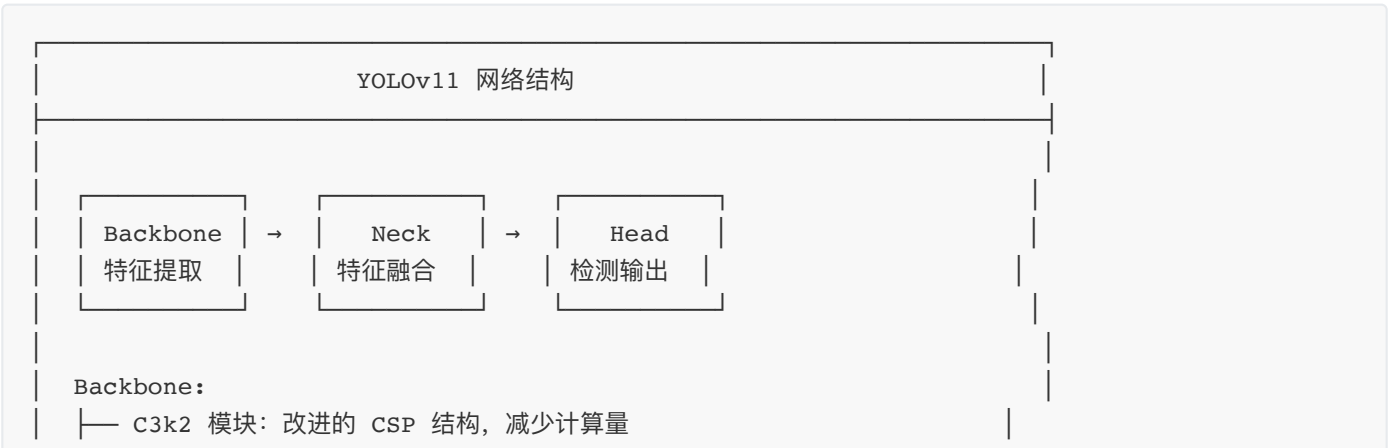
二、YOLOv11 原理速通

2.1 YOLO 系列演进

YOLO（You Only Look Once）是单阶段目标检测算法的代表，其核心思想是将目标检测视为一个端到端的回归问题——一次前向传播即可预测所有目标的类别和位置。



2.2 YOLOv11 网络结构概览



- └ SPPF (空间金字塔池化) : 增强多尺度感知
 - └ C2PSA (注意力机制) : 提升特征表达能力
- Neck:
- └ FPN (特征金字塔网络) : 自顶向下融合
 - └ PAN (路径聚合网络) : 自底向上融合
- Head:
- └ 解耦头: 分类和回归分开预测
 - └ Anchor-Free: 无需预设锚框

策略: Day 5 不需要理解网络内部细节。只需知道 YOLOv11 输入一张图像, 输出所有目标的类别和位置。训练时, 它需要特定格式的标注数据。

2.3 YOLOv11 输入输出

模型输入

输入: 一张 RGB 图像 (自动缩放到 640×640)
格式: BGR (OpenCV) 或 RGB (Pillow)
支持: .jpg, .png, .bmp, .tif 等常见图像格式

|



模型输出

对于每个检测到的目标:

- └ class_id - 类别编号 (从 0 开始)
- └ confidence - 置信度 (0~1, 越高越确定)
- └ bbox - 边界框 [x1, y1, x2, y2] (像素坐标)

示例输出:

```
[
  {class_id: 0, confidence: 0.92, bbox: [120, 80, 340, 260]},
  {class_id: 1, confidence: 0.87, bbox: [400, 150, 580, 390]}
]
```

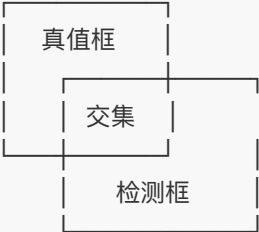
2.4 核心评估指标

目标检测评估指标

目标检测评估指标	
Precision (精确率)	精确率 = $TP / (TP + FP)$ "检测出的目标中, 有多少是真目标" 例: 检测出 10 个目标, 9 个正确 → Precision = 0.9

Recall (召回率)	召回率 = $TP / (TP + FN)$ "所有真目标中，有多少被检测出来" 例：图中有 12 个目标，检出 10 个 → $Recall = 0.833$
mAP50	在 $IoU=0.5$ 时的平均精度均值 "当检测框与真值框重叠 50% 以上时，各类别的平均精度" 最常用的指标，范围 0~1，越高越好
mAP50-95	在 IoU 从 0.5 到 0.95（步长 0.05）的平均精度均值 更严格的评估标准，要求检测框更精确
IoU (交并比)	$Intersection\ over\ Union$ （交并比） 检测框与真值框的重叠程度 $IoU = \text{交集面积} / \text{并集面积}$

IoU 可视化解理解：



$IoU = \text{交集面积} / \text{并集面积}$

$IoU > 0.5 \rightarrow$ 检测正确（一般标准）

$IoU > 0.75 \rightarrow$ 检测精确（严格标准）

2.5 YOLOv11 模型规格

模型	参数量	推理速度	mAP50-95	适用场景
YOLOv11n	2.6M	最快	39.5	移动端/嵌入式、快速原型
YOLOv11s	9.4M	快	43.0	轻量级部署
YOLOv11m	20.1M	中等	46.7	精度与速度平衡
YOLOv11l	25.3M	较慢	47.5	高精度需求
YOLOv11x	56.9M	最慢	48.5	极致精度

建议：Day 5~7 训练阶段统一使用 **YOLOv11n**（最小模型），训练速度快，适合快速验证。模型效果调优后再考虑升级。

三、YOLO 标注格式详解

3.1 YOLO TXT 格式规范

YOLO 系列使用纯文本文件作为标注格式，每张图像对应一个同名的 `.txt` 文件：

YOLO 标注格式

文件名：与图像同名（如 `image001.jpg` → `image001.txt`）
每行一个目标： `class_id x_center y_center width height`

所有坐标值均为 归一化到 `[0, 1]` 的浮点数：

- `x_center` = 目标中心点 `x` 坐标 / 图像宽度
- `y_center` = 目标中心点 `y` 坐标 / 图像高度
- `width` = 目标宽度 / 图像宽度
- `height` = 目标高度 / 图像高度

`class_id`：从 0 开始的整数类别编号

3.2 标注示例

假设图像尺寸为 `800×600`，图中有两个目标：

目标 1：飞机 (`class_id=0`)
边界框像素坐标： `xmin=100, ymin=80, xmax=420, ymax=320`

目标 2：油罐 (`class_id=1`)
边界框像素坐标： `xmin=500, ymin=200, xmax=700, ymax=450`

转换计算过程：

目标 1 (飞机, `class_id=0`) :

$$\begin{aligned}x_center &= (100 + 420) / 2 / 800 = 260 / 800 = 0.325000 \\y_center &= (80 + 320) / 2 / 600 = 200 / 600 = 0.333333 \\width &= (420 - 100) / 800 = 320 / 800 = 0.400000 \\height &= (320 - 80) / 600 = 240 / 600 = 0.400000\end{aligned}$$

→ 标注行： `0 0.325000 0.333333 0.400000 0.400000`

目标 2 (油罐, `class_id=1`) :

$$\begin{aligned}x_center &= (500 + 700) / 2 / 800 = 600 / 800 = 0.750000 \\y_center &= (200 + 450) / 2 / 600 = 325 / 600 = 0.541667 \\width &= (700 - 500) / 800 = 200 / 800 = 0.250000 \\height &= (450 - 200) / 600 = 250 / 600 = 0.416667\end{aligned}$$

→ 标注行： `1 0.750000 0.541667 0.250000 0.416667`

最终标注文件 `image001.txt`：

```
0 0.325000 0.333333 0.400000 0.400000
1 0.750000 0.541667 0.250000 0.416667
```

3.3 坐标转换公式汇总

像素坐标 → YOLO 归一化坐标

已知: x_{min} , y_{min} , x_{max} , y_{max} , img_width , img_height

```
x_center = (x_min + x_max) / 2.0 / img_width
y_center = (y_min + y_max) / 2.0 / img_height
width     = (x_max - x_min) / img_width
height    = (y_max - y_min) / img_height
```

YOLO 归一化坐标 → 像素坐标 (反向转换)

```
x_min = (x_center - width/2) * img_width
y_min = (y_center - height/2) * img_height
x_max = (x_center + width/2) * img_width
y_max = (y_center + height/2) * img_height
```

3.4 格式要求注意事项

YOLO 标注文件规范

- ✓ 每行一个目标, 空格分隔 5 个值
- ✓ `class_id` 是整数 (0, 1, 2, ...), 不带小数点
- ✓ 坐标值保留 6 位小数 (精度足够且文件不过大)
- ✓ 坐标值范围 $[0, 1]$, 超出范围说明标注有误
- ✓ 空图像 (无目标) 对应空 `.txt` 文件 (文件存在但内容为空)
- ✓ 文件名必须与图像文件名完全一致 (不含扩展名)
- ✗ 不要有表头行
- ✗ 不要有逗号分隔 (用空格)
- ✗ 不要使用像素坐标 (必须归一化)

四、常见标注格式对比

在获取公开数据集或使用标注工具时, 会遇到多种标注格式。理解它们的差异是格式转换的基础。

4.1 四大主流格式对比

目标检测标注格式对比				
特性	YOLO TXT	VOC XML	COCO JSON	LabelMe JSON
文件类型	纯文本 .txt	XML .xml	JSON .json	JSON .json
文件数量	每张图一个	每张图一个	整个数据集一个文件	每张图一个
坐标格式	归一化 x_c y_c w h	像素坐标 xmin ymin xmax ymax	像素坐标 x y w h	像素坐标 点列表
类别编号	从 0 开始 整数	类名字符串	category_id 整数	类名字符串
图像信息	不含	含宽高	含宽高	含宽高
标注工具	原生支持	LabelImg	CVAT等	LabelMe
常见来源	Ultralytics Roboflow	Pascal VOC ImageNet	MS COCO Kaggle	自建标注

4.2 各格式示例

VOC XML 格式（每张图一个 `.xml` 文件）：

```
<annotation>
  <filename>image001.jpg</filename>
  <size>
    <width>800</width>
    <height>600</height>
    <depth>3</depth>
  </size>
  <object>
    <name>airplane</name>
    <bndbox>
      <xmin>100</xmin>
      <ymin>80</ymin>
      <xmax>420</xmax>
      <ymax>320</ymax>
    </bndbox>
  </object>
  <object>
    <name>storage-tank</name>
    <bndbox>
      <xmin>500</xmin>
      <ymin>200</ymin>
      <xmax>700</xmax>
      <ymax>450</ymax>
    </bndbox>
  </object>
</annotation>
```



```
</bndbox>
</object>
</annotation>
```

COCO JSON 格式（整个数据集一个 `.json` 文件）：

```
{
  "images": [
    { "id": 1, "file_name": "image001.jpg", "width": 800, "height": 600 }
  ],
  "annotations": [
    {
      "id": 1,
      "image_id": 1,
      "category_id": 1,
      "bbox": [100, 80, 320, 240],
      "area": 76800
    },
    {
      "id": 2,
      "image_id": 1,
      "category_id": 2,
      "bbox": [500, 200, 200, 250],
      "area": 50000
    }
  ],
  "categories": [
    { "id": 1, "name": "airplane" },
    { "id": 2, "name": "storage-tank" }
  ]
}
```

注意：COCO 的 `bbox` 格式是 `[x_min, y_min, width, height]`，不是 `[xmin, ymin, xmax, ymax]`。

LabelMe JSON 格式（每张图一个 `.json` 文件）：

```
{
  "version": "5.0.0",
  "imagePath": "image001.jpg",
  "imageWidth": 800,
  "imageHeight": 600,
  "shapes": [
    {
      "label": "airplane",
      "shape_type": "rectangle",
      "points": [
        [100, 80],
        [420, 320]
      ]
    }
  ],
  {
```

```
    "label": "storage-tank",
    "shape_type": "rectangle",
    "points": [
      [500, 200],
      [700, 450]
    ]
  }
]
```

五、数据集获取渠道与实操

5.1 公开数据集平台

数据集获取平台一览	
Kaggle	https://www.kaggle.com/datasets 优势：数据集丰富、可在线预览、下载方便 搜索关键词：object detection + 场景名
Roboflow Universe	https://universe.roboflow.com/ 优势：直接导出 YOLO 格式、自带数据增强 推荐：大部分场景首选此平台
Google Dataset	https://datasetsearch.research.google.com/ 优势：聚合搜索引擎，覆盖广
Papers with Code	https://paperswithcode.com/datasets 优势：与论文关联、有 benchmark 对比
各场景官方数据集	见下方各场景推荐列表

5.2 各场景推荐数据集

智慧农业（1-3 组）：

数据集	来源	检测目标	数据量	标注格式
PlantVillage	Kaggle	作物叶片病害	54,000+	分类（需转检测）
Tomato Detection	Roboflow	番茄成熟度	3,000+	YOLO/COCO
Weed Detection	Roboflow	杂草识别	2,000+	YOLO
Cotton Disease	Kaggle	棉花病害	1,500+	VOC

工业检测（4-6 组）：

数据集	来源	检测目标	数据量	标注格式
NEU-DET	东北大学	钢材表面缺陷（6类）	1,800	VOC
KolektorSDD	官方	电子元件缺陷	399	VOC
DAGM 2007	官方	纹理表面缺陷	11,000+	分类（需标注）
PCB Defect	Roboflow	PCB 板缺陷	2,000+	YOLO

遥感检测（7-9 组）：

数据集	来源	检测目标	数据量	标注格式
DOTA	官方	15 类遥感目标	2,800+	DOTA TXT
DIOR	官方	20 类遥感目标	23,000+	VOC XML
RSD	GitHub	遥感场景	10,000+	VOC
DOTA-v1.5	Roboflow	飞机/油罐/桥梁等	子集可选	YOLO

医疗影像（10-12 组）：

数据集	来源	检测目标	数据量	标注格式
Chest X-ray	Kaggle	肺部异常	5,800+	COCO
Brain Tumor	Kaggle	脑部肿瘤	3,000+	YOLO
Blood Cells	Roboflow	血细胞分类	12,000+	YOLO/COCO

交通/安防（13-14 组）：

数据集	来源	检测目标	数据量	标注格式
UA-DETRAC	官方	车辆检测跟踪	140,000+	VOC
WIDER FACE	官方	人脸检测	390,000+	自定义 TXT
BDD100K	官方	交通场景	100,000+	COCO

5.3 Roboflow 获取数据集实操（推荐方式）

Roboflow 是最推荐的数据获取方式，因为它可以直接导出 YOLO 格式，省去格式转换步骤。

步骤 1：访问 Roboflow Universe

浏览器打开 <https://universe.roboflow.com/>，使用 Google/GitHub 账号登录。

步骤 2：搜索数据集

在搜索框中输入场景关键词，例如：

- `airplane detection satellite` — 遥感飞机检测
- `surface defect metal` — 金属表面缺陷
- `crop disease leaf` — 作物病害叶片
- `brain tumor mri` — 脑部肿瘤 MRI

步骤 3：选择并下载

1. 点击进入目标数据集页面
2. 查看数据集信息：图像数量、类别数、标注质量
3. 点击 **"Download Dataset"**
4. 在 **Format** 下拉框中选择 **"YOLOv11"**
5. 选择下载方式：
 - **Download Zip** — 直接下载 ZIP 文件（小数据集推荐）
 - **Show download code** — 获取 Python 代码片段（大数据集推荐）

步骤 4：解压并组织目录

```
# 创建数据集目录
cd rsod-agent-platform
mkdir -p datasets

# 解压下载的 ZIP 到对应场景目录
# 假设下载了遥感检测数据集
unzip ~/Downloads/dataset.zip -d datasets/rsod/

# Roboflow 导出的 YOLOv11 格式已包含正确目录结构：
# datasets/rsod/
# |— train/
# |   |— images/
# |   |— labels/
# |   └— data.yaml （可能位于根目录）
# |— valid/
# |   |— images/
# |   └— labels/
# └— test/
#     |— images/
#     └— labels/
```

注意：Roboflow 导出的目录结构可能将 `images` 和 `labels` 放在同一级目录下，需要按第十节的规范进行重组。

5.4 Kaggle 获取数据集实操

步骤 1：安装 Kaggle CLI

```
cd backend
source .venv/bin/activate
pip install kaggle
```

步骤 2: 配置 Kaggle API Key

1. 登录 <https://www.kaggle.com>
2. 点击头像 → Settings → API → Create New Token
3. 下载 `kaggle.json` 文件
4. 将其放到 `~/.kaggle/kaggle.json` :

```
mkdir -p ~/.kaggle
cp ~/Downloads/kaggle.json ~/.kaggle/kaggle.json
chmod 600 ~/.kaggle/kaggle.json
```

步骤 3: 搜索并下载数据集

```
# 搜索数据集
kaggle datasets list -s "satellite airplane detection"

# 下载指定数据集
kaggle datasets download -d <owner>/<dataset-name> -p datasets/

# 解压
cd datasets
unzip <dataset-name>.zip
rm <dataset-name>.zip
```

注意: Kaggle 数据集标注格式不统一, 下载后需先检查标注格式, 再使用 DataConverter 转换。

六、数据标注工具使用教学

当公开数据集无法满足需求, 或需要为自有图像添加标注时, 使用以下工具。

6.1 LabelImg (VOC 格式标注)

LabelImg 是经典的目标检测标注工具, 默认输出 VOC XML 格式。

安装:

```
cd backend
source .venv/bin/activate
pip install labelImg
```

启动:

labelImg

使用流程：

LabelImg 标注流程

1. 打开图像目录
→ File → Open Dir → 选择图像文件夹
2. 设置标注输出目录
→ Change Save Dir → 选择标注文件保存目录
3. 选择标注格式
→ 左侧面板 → 选择 PascalVOC (默认) 或 YOLO
→ 建议选 PascalVOC, 后续统一用 DataConverter 转换
4. 标注操作
→ 按 W 键创建矩形框
→ 拖拽绘制边界框
→ 输入类别名称 (如 airplane、storage-tank)
→ 按 D 键切换到下一张图片
→ 按 A 键切换到上一张图片
5. 快捷键
 - ├─ W — 创建矩形框
 - ├─ D — 下一张
 - ├─ A — 上一张
 - ├─ Ctrl+S — 保存当前标注
 - └─ Del — 删除选中的框

6.2 LabelMe (JSON 格式标注)

LabelMe 功能更强大, 支持多边形、圆形等多种标注形状, 输出 JSON 格式。

安装：

```
cd backend
source .venv/bin/activate
pip install labelme
```

启动：

labelme

使用流程：

LabelMe 标注流程

1. 打开图像目录

→ File → Open Dir → 选择图像文件夹

2. 选择标注模式

→ 目标检测使用 "Create Rectangle" (矩形框)

→ 实例分割使用 "Create Polygon" (多边形)

3. 标注操作

→ 点击 "Create Rectangle"

→ 在图像上拖拽绘制矩形框

→ 输入类别标签 (如 airplane、storage-tank)

→ 按 D 键下一张 / A 键上一张

4. 保存

→ File → Save (保存当前标注)

→ 或开启 "Save Automatically" (自动保存)

→ 每张图像生成一个同名 .json 文件

5. 输出文件

→ 每张图像对应一个 .json 文件

→ JSON 中包含: 图像路径、宽高、所有标注形状

6.3 LabelImg vs LabelMe 选择建议

特性	LabelImg	LabelMe
标注类型	仅矩形框	矩形框 + 多边形 + 圆形
输出格式	VOC XML / YOLO TXT	JSON
上手难度	简单	中等
适用场景	纯目标检测	目标检测 + 实例分割
建议	检测任务首选	需要多边形标注时使用

推荐：如果从零开始标注，建议用 LabelImg 并选择 PascalVOC 格式，后续统一用 DataConverter 转换为 YOLO 格式。

七、DataConverter 实现：VOC XML → YOLO TXT

7.1 VOC XML 结构解析

VOC XML 结构解析

```

<annotation>
├── <filename>      - 图像文件名
├── <size>
│   ├── <width>     - 图像宽度 (像素)
│   ├── <height>    - 图像高度 (像素)
│   └── <depth>     - 通道数 (RGB=3)
└── <object>       - 可重复, 每个 object 代表一个目标
    ├── <name>      - 类别名称 (字符串)
    ├── <difficult> - 是否为困难样本 (0/1)
    └── <bndbox>
        ├── <xmin>  - 左上角 x (像素)
        ├── <ymin>  - 左上角 y (像素)
        ├── <xmax>  - 右下角 x (像素)
        └── <ymax>  - 右下角 y (像素)

```

7.2 创建 training 模块目录

```

cd backend/app
mkdir -p training
touch training/__init__.py

```

7.3 实现 VOC → YOLO 转换

RSOD 数据集原始格式为 Pascal VOC (XML标注)，需要转换为 YOLO 格式：

原始格式 (Pascal VOC) :

```

rsod/
├── images/
│   ├── aircraft_001.jpg
│   ├── aircraft_002.jpg
│   └── ...
└── annotations/
    ├── aircraft_001.xml
    ├── aircraft_002.xml
    └── ...

```

目标格式 (YOLO) :

```

rsod/yolo_dataset/
├── images/
│   ├── train/
│   │   ├── aircraft_001.jpg
│   │   └── ...
│   └── val/
│       ├── aircraft_xxx.jpg
│       └── ...
├── labels/
│   ├── train/
│   │   ├── aircraft_001.txt
│   │   └── ...

```



```
|   └─ val/
|       └─ aircraft_xxx.txt
|           └─ ...
└─ data.yaml
```

文件: `backend/app/training/data_converter.py`

```
"""
```

数据集格式转换器

职责:

- 将 VOC XML 格式转换为 YOLO TXT 格式
- 将 COCO JSON 格式转换为 YOLO TXT 格式
- 将 LabelMe JSON 格式转换为 YOLO TXT 格式
- 所有转换均输出 YOLO 归一化坐标格式

使用方式:

```
from app.training.data_converter import DataConverter
```

```
converter = DataConverter()
```

```
converter.voc_to_yolo(
    xml_dir="datasets/rsod/raw/annotations",
    output_dir="datasets/rsod/labels/train",
    class_mapping={"airplane": 0, "storage-tank": 1}
)
```

```
"""
```

```
import json
```

```
import os
```

```
import xml.etree.ElementTree as ET
```

```
from pathlib import Path
```

```
from app.core.logger import get_logger
```

```
logger = get_logger(__name__)
```

```
class DataConverter:
```

```
    """数据集格式转换器 - 将各种标注格式统一转换为 YOLO TXT 格式"""
```

```
    @staticmethod
```

```
    def voc_to_yolo(xml_dir: str, output_dir: str, class_mapping: dict) -> dict:
```

```
        """
```

VOC XML 格式 → YOLO TXT 格式

转换逻辑:

1. 遍历 xml_dir 下所有 .xml 文件
2. 解析每个 XML 获取图像尺寸和目标边界框
3. 将像素坐标转换为 YOLO 归一化坐标
4. 输出到 output_dir, 每张图一个 .txt 文件

Args:

xml_dir: VOC XML 标注文件目录
output_dir: YOLO TXT 输出目录
class_mapping: 类别映射字典 {类名字符串: 类别ID整数}
例如 {"airplane": 0, "storage-tank": 1}

Returns:

转换统计信息字典 {"total": 总数, "converted": 成功数, "skipped": 跳过数}
"""

```
os.makedirs(output_dir, exist_ok=True)

stats = {"total": 0, "converted": 0, "skipped": 0, "errors": []}

xml_files = list(Path(xml_dir).glob("*.xml"))
if not xml_files:
    logger.warning("VOC 转换: 目录 %s 下未找到 .xml 文件", xml_dir)
    return stats

logger.info("VOC 转换开始: %d 个 XML 文件 → %s", len(xml_files), output_dir)

for xml_file in xml_files:
    stats["total"] += 1
    try:
        tree = ET.parse(xml_file)
        root = tree.getroot()

        # 获取图像尺寸
        size = root.find("size")
        if size is None:
            logger.warning("XML %s 缺少 <size> 标签, 跳过", xml_file.name)
            stats["skipped"] += 1
            continue

        width_elem = size.find("width")
        height_elem = size.find("height")
        if width_elem is None or height_elem is None:
            logger.warning(
                "XML %s 缺少 width 或 height 标签, 跳过", xml_file.name
            )
            stats["skipped"] += 1
            continue

        if width_elem.text is None or height_elem.text is None:
            logger.warning(
                "XML %s 的 width 或 height 文本为空, 跳过", xml_file.name
            )
            stats["skipped"] += 1
            continue

        img_width = int(width_elem.text)
        img_height = int(height_elem.text)
```

```

if img_width <= 0 or img_height <= 0:
    logger.warning(
        "XML %s 图像尺寸无效 (%d, %d), 跳过",
        xml_file.name,
        img_width,
        img_height,
    )
    stats["skipped"] += 1
    continue

# 解析所有目标
yolo_lines = []
for obj in root.findall("object"):
    name_elem = obj.find("name")
    if name_elem is None or name_elem.text is None:
        continue

    class_name = name_elem.text.strip()
    if class_name not in class_mapping:
        logger.warning(
            "XML %s 中类别 '%s' 不在映射表中, 跳过该目标",
            xml_file.name,
            class_name,
        )
        continue

    class_id = class_mapping[class_name]
    bbox = obj.find("bndbox")
    if bbox is None:
        continue

    xmin_elem = bbox.find("xmin")
    ymin_elem = bbox.find("ymin")
    xmax_elem = bbox.find("xmax")
    ymax_elem = bbox.find("ymax")

    if (
        xmin_elem is None
        or ymin_elem is None
        or xmax_elem is None
        or ymax_elem is None
    ):
        continue

    if (
        xmin_elem.text is None
        or ymin_elem.text is None
        or xmax_elem.text is None
        or ymax_elem.text is None
    ):
        continue

```

```

xmin = float(xmin_elem.text)
ymin = float(ymin_elem.text)
xmax = float(xmax_elem.text)
ymax = float(ymax_elem.text)

# 边界值裁剪: 确保坐标不超出图像范围
xmin = max(0, min(xmin, img_width))
ymin = max(0, min(ymin, img_height))
xmax = max(0, min(xmax, img_width))
ymax = max(0, min(ymax, img_height))

# 过滤无效框 (宽或高为 0)
if xmax <= xmin or ymax <= ymin:
    logger.warning(
        "XML %s 中目标 '%s' 边界框无效, 跳过",
        xml_file.name,
        class_name,
    )
    continue

# 像素坐标 → YOLO 归一化坐标
x_center = (xmin + xmax) / 2.0 / img_width
y_center = (ymin + ymax) / 2.0 / img_height
width = (xmax - xmin) / img_width
height = (ymax - ymin) / img_height

yolo_lines.append(
    f"{class_id} {x_center:.6f} {y_center:.6f} "
    f"{width:.6f} {height:.6f}"
)

# 保存 YOLO 标注文件
txt_file = Path(output_dir) / f"{xml_file.stem}.txt"
with open(txt_file, "w", encoding="utf-8") as f:
    f.write("\n".join(yolo_lines))

stats["converted"] += 1

except ET.ParseError as e:
    logger.error("XML 解析失败 %s: %s", xml_file.name, str(e))
    stats["errors"].append(str(xml_file.name))
    stats["skipped"] += 1
except Exception as e:
    logger.error("VOC 转换异常 %s: %s", xml_file.name, str(e))
    stats["errors"].append(str(xml_file.name))
    stats["skipped"] += 1

logger.info(
    "VOC 转换完成: 总计 %d, 成功 %d, 跳过 %d",
    stats["total"],
    stats["converted"],
    stats["skipped"],

```

```
)  
return stats
```

关键知识点讲解：

1. `xml.etree.ElementTree`：Python 标准库的 XML 解析器，通过 `parse()` 读取 XML，用 `find()` 和 `findall()` 按标签名查找节点
2. `class_mapping` 参数：将 VOC 的字符串类别名（如 "airplane"）映射为 YOLO 的整数编号（如 0），由调用方根据 data.yaml 中的类别顺序指定
3. 边界值裁剪：`max(0, min(x, img_width))` 确保坐标不超出图像边界，某些标注工具的框可能略微超出图像范围
4. 统计信息返回：返回转换统计字典，方便调用方了解转换结果，也便于在 API 接口中返回给前端

7.4 测试 VOC 转换

创建测试脚本：`tools/convert_voc.py`

```
#!/usr/bin/env python3  
"""  
VOC XML → YOLO TXT 数据集格式转换脚本  
  
功能：  
1. 检查原始数据集中图片与XML标注文件的匹配度，删除不匹配的文件  
2. 将VOC格式的XML标注文件转换为YOLO格式的TXT标注文件  
3. 按8:1:1比例划分训练集/验证集/测试集  
4. 生成YOLO训练所需的data.yaml配置文件  
  
使用方式：  
cd rsod-agent-platform/backend  
python ../tools/convert_voc.py  
  
处理流程：  
raw/images + raw/annotations → temp_labels → yolo_dataset/  
├─ images/{train,val,test}/  
├─ labels/{train,val,test}/  
└─ data.yaml  
  
数据集：RSOD (Remote Sensing Object Detection)  
类别：aircraft, oiltank, overpass, playground  
"""  
  
import os  
import random  
import shutil  
import xml.etree.ElementTree as ET  
  
# 项目根目录路径 (tools/convert_voc.py → rsod-agent-platform/)  
PROJECT_ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))  
  
# 原始数据目录配置
```

```

RAW_IMAGE_DIR = os.path.join(PROJECT_ROOT, "datasets/rsod/raw/images")
RAW_ANNOTATION_DIR = os.path.join(PROJECT_ROOT, "datasets/rsod/raw/annotations")

# YOLO格式输出目录
OUTPUT_DIR = os.path.join(PROJECT_ROOT, "datasets/rsod/yolo_dataset")

# 数据集类别配置
CLASS_NAMES = ["aircraft", "oiltank", "overpass", "playground"]
CLASS_MAPPING = {name: idx for idx, name in enumerate(CLASS_NAMES)}

# 支持的图片扩展名
IMAGE_EXTS = {".jpg", ".jpeg", ".png", ".bmp", ".tif", ".tiff"}

def voc_to_yolo(xml_dir: str, output_dir: str, class_mapping: dict) -> dict:
    """
    将VOC格式的XML标注文件批量转换为YOLO格式的TXT标注文件

    参数:
        xml_dir: VOC XML标注文件所在目录
        output_dir: YOLO TXT标注文件输出目录
        class_mapping: 类别名称到数字ID的映射字典

    返回:
        转换统计信息字典 {"total": 总数, "converted": 成功数, "skipped": 跳过数}

    YOLO标注格式 (每行一个目标):
        class_id x_center y_center width height
        其中坐标为归一化值 (0~1)

    VOC标注格式 (XML中的bndbox):
        <xmin> <ymin> <xmax> <ymax>
        其中坐标为像素值

    转换公式:
        x_center = (xmin + xmax) / 2.0 / image_width
        y_center = (ymin + ymax) / 2.0 / image_height
        width = (xmax - xmin) / image_width
        height = (ymax - ymin) / image_height
    """
    # 创建输出目录 (如果已存在则跳过)
    os.makedirs(output_dir, exist_ok=True)

    # 初始化统计信息
    stats = {"total": 0, "converted": 0, "skipped": 0}

    # 获取所有XML文件
    xml_files = [f for f in os.listdir(xml_dir) if f.lower().endswith(".xml")]
    if not xml_files:
        print(" 警告: 目录下未找到 .xml 文件")
        return stats

```

```

# 遍历处理每个XML文件
for xml_file in xml_files:
    stats["total"] += 1
    xml_path = os.path.join(xml_dir, xml_file)

    try:
        # 解析XML文件
        tree = ET.parse(xml_path)
        root = tree.getroot()

        # 获取图片尺寸信息
        size = root.find("size")
        if size is None:
            stats["skipped"] += 1
            continue

        width_elem = size.find("width")
        height_elem = size.find("height")
        if width_elem is None or height_elem is None:
            stats["skipped"] += 1
            continue

        if width_elem.text is None or height_elem.text is None:
            stats["skipped"] += 1
            continue

        img_width = int(width_elem.text)
        img_height = int(height_elem.text)

        # 检查尺寸有效性
        if img_width <= 0 or img_height <= 0:
            stats["skipped"] += 1
            continue

        # 解析所有目标对象
        yolo_lines = []
        for obj in root.findall("object"):
            # 获取类别名称
            name_elem = obj.find("name")
            if name_elem is None or name_elem.text is None:
                continue

            class_name = name_elem.text.strip()
            if class_name not in class_mapping:
                continue

            class_id = class_mapping[class_name]

            # 获取边界框坐标
            bbox = obj.find("bndbox")
            if bbox is None:
                continue

```

```

xmin_elem = bbox.find("xmin")
ymin_elem = bbox.find("ymin")
xmax_elem = bbox.find("xmax")
ymax_elem = bbox.find("ymax")

# 检查边界框元素是否完整
if (
    xmin_elem is None
    or ymin_elem is None
    or xmax_elem is None
    or ymax_elem is None
):
    continue

# 检查边界框文本是否完整
if (
    xmin_elem.text is None
    or ymin_elem.text is None
    or xmax_elem.text is None
    or ymax_elem.text is None
):
    continue

# 将坐标转换为浮点型
xmin = float(xmin_elem.text)
ymin = float(ymin_elem.text)
xmax = float(xmax_elem.text)
ymax = float(ymax_elem.text)

# 边界值裁剪：确保坐标不超出图像范围
# 处理截断目标 (truncated=1) 的越界坐标
xmin = max(0, min(xmin, img_width))
ymin = max(0, min(ymin, img_height))
xmax = max(0, min(xmax, img_width))
ymax = max(0, min(ymax, img_height))

# 过滤无效框 (宽或高 <= 0)
if xmax <= xmin or ymax <= ymin:
    continue

# VOC像素坐标 → YOLO归一化坐标
x_center = (xmin + xmax) / 2.0 / img_width
y_center = (ymin + ymax) / 2.0 / img_height
width = (xmax - xmin) / img_width
height = (ymax - ymin) / img_height

# 格式化输出 (保留6位小数)
yolo_lines.append(
    f"{class_id} {x_center:.6f} {y_center:.6f} {width:.6f} {height:.6f}"
)

```



```

        # 如果有有效标注, 写入txt文件
        if yolo_lines:
            basename = os.path.splitext(xml_file)[0]
            output_path = os.path.join(output_dir, f"{basename}.txt")
            with open(output_path, "w", encoding="utf-8") as f:
                f.write("\n".join(yolo_lines))
            stats["converted"] += 1
        else:
            stats["skipped"] += 1

    except Exception:
        stats["skipped"] += 1

    return stats


def main():
    """主函数: 执行完整的数据集处理流程"""
    print("=" * 70)
    print("      RSOD 数据集处理流程")
    print("=" * 70)

    # — 步骤1: 检查图片与XML匹配度 —
    print("\n[1] 检查图片与XML匹配度")

    # 获取所有图片文件名 (不含扩展名)
    image_stems = set()
    for f in os.listdir(RAW_IMAGE_DIR):
        if os.path.isfile(os.path.join(RAW_IMAGE_DIR, f)):
            ext = os.path.splitext(f)[1].lower()
            if ext in IMAGE_EXTS:
                image_stems.add(os.path.splitext(f)[0])

    # 获取所有XML文件名 (不含扩展名)
    xml_stems = set()
    for f in os.listdir(RAW_ANNOTATION_DIR):
        if os.path.isfile(os.path.join(RAW_ANNOTATION_DIR, f)):
            if f.lower().endswith(".xml"):
                xml_stems.add(os.path.splitext(f)[0])

    print(f"  图片数: {len(image_stems)}, XML数: {len(xml_stems)}")

    # 找出不匹配的文件并删除
    images_without_xml = image_stems - xml_stems # 有图片无标注
    xmls_without_image = xml_stems - image_stems # 有标注无图片

    for stem in images_without_xml:
        for ext in IMAGE_EXTS:
            img_file = os.path.join(RAW_IMAGE_DIR, f"{stem}{ext}")
            if os.path.exists(img_file):
                os.remove(img_file)

```

```

for stem in xmls_without_image:
    xml_file = os.path.join(RAW_ANNOTATION_DIR, f"{stem}.xml")
    if os.path.exists(xml_file):
        os.remove(xml_file)

# — 步骤2: VOC转YOLO格式 —
print("\n[2] VOC转YOLO格式")

# 创建临时标注目录（用于存放转换后的TXT文件）
temp_label_dir = os.path.join(OUTPUT_DIR, "temp_labels")
os.makedirs(temp_label_dir, exist_ok=True)

# 执行转换
stats = voc_to_yolo(RAW_ANNOTATION_DIR, temp_label_dir, CLASS_MAPPING)
print(
    f"  转换结果: 总计 {stats['total']}, 成功 {stats['converted']}, 跳过 {stats['skipped']}"
)

# — 步骤3: 划分数据集 —
print("\n[3] 划分数据集")

# 获取所有图片文件
image_files = []
for f in os.listdir(RAW_IMAGE_DIR):
    if os.path.isfile(os.path.join(RAW_IMAGE_DIR, f)):
        ext = os.path.splitext(f)[1].lower()
        if ext in IMAGE_EXTS:
            image_files.append(f)

# 设置随机种子, 保证划分结果可重复
random.seed(42)
random.shuffle(image_files)

# 按8:1:1比例划分
total = len(image_files)
train_end = int(total * 0.8) # 前80%为训练集
val_end = train_end + int(total * 0.1) # 中间10%为验证集
# 剩余10%为测试集

splits = {
    "train": image_files[:train_end],
    "val": image_files[train_end:val_end],
    "test": image_files[val_end:],
}

# 将图片和标注复制到对应目录
for split_name, files in splits.items():
    img_out = os.path.join(OUTPUT_DIR, "images", split_name)
    lbl_out = os.path.join(OUTPUT_DIR, "labels", split_name)
    os.makedirs(img_out, exist_ok=True)
    os.makedirs(lbl_out, exist_ok=True)

```

```

for filename in files:
    # 复制图片
    src_image = os.path.join(RAW_IMAGE_DIR, filename)
    dst_image = os.path.join(img_out, filename)
    shutil.copy2(src_image, dst_image)

    # 复制标注 (如果存在)
    basename = os.path.splitext(filename)[0]
    label_file = os.path.join(temp_label_dir, f"{basename}.txt")
    if os.path.exists(label_file):
        shutil.copy2(label_file, os.path.join(lbl_out, f"{basename}.txt"))
    else:
        # 创建空标注文件
        open(os.path.join(lbl_out, f"{basename}.txt"), "w").close()

print(f" {split_name}: {len(files)} 个")

# — 步骤4: 生成data.yaml配置文件 —
print("\n[4] 生成data.yaml")

# 构建YAML内容
yaml_content = f"""path: {os.path.basename(OUTPUT_DIR)}
train: images/train
val: images/val
test: images/test
nc: {len(CLASS_NAMES)}
names:
"""

for i, name in enumerate(CLASS_NAMES):
    yaml_content += f" {i}: {name}\n"

# 写入YAML文件
yaml_path = os.path.join(OUTPUT_DIR, "data.yaml")
with open(yaml_path, "w", encoding="utf-8") as f:
    f.write(yaml_content)
print(f" 配置文件已生成: {yaml_path}")

# 清理临时目录
shutil.rmtree(temp_label_dir, ignore_errors=True)

# 输出完成信息
print("\n" + "=" * 70)
print(f" 处理完成! 输出目录: {OUTPUT_DIR}")
print("=" * 70)

if __name__ == "__main__":
    # 脚本直接运行时执行主函数
    main()

```

八、DataConverter 实现：COCO JSON → YOLO TXT

8.1 COCO JSON 结构解析

COCO JSON 结构解析

```
{
  "images": [           - 所有图像信息
    {
      "id": 1,           - 图像唯一 ID
      "file_name": "image001.jpg",
      "width": 800,      - 图像宽度 (像素)
      "height": 600     - 图像高度 (像素)
    }, ...
  ],
  "annotations": [     - 所有标注信息
    {
      "id": 1,           - 标注唯一 ID
      "image_id": 1,     - 对应的图像 ID
      "category_id": 1,  - 类别 ID
      "bbox": [x, y, w, h], ← 注意: 左上角x, 左上角y, 宽, 高
      "area": 76800,     - 目标面积
      "iscrowd": 0       - 是否为密集群体
    }, ...
  ],
  "categories": [       - 类别定义
    {
      "id": 1,           - 类别 ID
      "name": "airplane" - 类别名称
    }, ...
  ]
}
```

重要: COCO bbox 是 [x_min, y_min, width, height]
不是 [xmin, ymin, xmax, ymax]!

8.2 实现 COCO → YOLO 转换

文件: backend/app/training/data_converter.py (在 DataConverter 类中添加方法)

```
@staticmethod
def coco_to_yolo(
    json_file: str, output_dir: str, class_mapping: dict = None
) -> dict:
    """
    COCO JSON 格式 → YOLO TXT 格式

    转换逻辑:
```

1. 读取整个 COCO JSON 文件
2. 构建 category_id → class_id 映射
3. 按 image_id 分组所有标注
4. 遍历每张图像, 将 COCO bbox 转换为 YOLO 归一化坐标
5. 每张图像输出一个 .txt 文件

Args:

json_file: COCO 标注 JSON 文件路径
output_dir: YOLO TXT 输出目录
class_mapping: 可选的类别重映射
 如果为 None, 则按 categories 中的顺序自动编号 (从 0 开始)
 如果提供, 格式为 {category_name: class_id}

Returns:

转换统计信息字典

```
"""
os.makedirs(output_dir, exist_ok=True)
stats = {"total": 0, "converted": 0, "skipped": 0, "errors": []}

# 读取 COCO JSON
try:
    with open(json_file, "r", encoding="utf-8") as f:
        coco_data = json.load(f)
except (json.JSONDecodeError, FileNotFoundError) as e:
    logger.error("COCO JSON 读取失败 %s: %s", json_file, str(e))
    stats["errors"].append(str(e))
    return stats

# 构建类别映射
if class_mapping is None:
    # 自动按 categories 顺序编号 (从 0 开始)
    category_mapping = {
        cat["id"]: idx
        for idx, cat in enumerate(coco_data.get("categories", []))
    }
    logger.info("COCO 自动类别映射: %s", category_mapping)
else:
    # 使用用户提供的映射: 需要先建立 coco_category_id → class_id 的关系
    cat_name_to_id = {
        cat["name"]: cat["id"] for cat in coco_data.get("categories", [])
    }
    category_mapping = {}
    for cat_name, class_id in class_mapping.items():
        if cat_name in cat_name_to_id:
            category_mapping[cat_name_to_id[cat_name]] = class_id
        else:
            logger.warning("COCO categories 中未找到类别 '%s'", cat_name)

# 按 image_id 分组标注
image_annotations = {}
for ann in coco_data.get("annotations", []):
    img_id = ann["image_id"]
```

```

        if img_id not in image_annotations:
            image_annotations[img_id] = []
        image_annotations[img_id].append(ann)

# 遍历每张图像
images = coco_data.get("images", [])
stats["total"] = len(images)
logger.info("COCO 转换开始: %d 张图像 → %s", len(images), output_dir)

for img_info in images:
    img_id = img_info["id"]
    img_width = img_info["width"]
    img_height = img_info["height"]
    file_name = img_info["file_name"]

    if img_width <= 0 or img_height <= 0:
        logger.warning("图像 %s 尺寸无效, 跳过", file_name)
        stats["skipped"] += 1
        continue

    yolo_lines = []
    for ann in image_annotations.get(img_id, []):
        # 检查类别是否在映射中
        cat_id = ann["category_id"]
        if cat_id not in category_mapping:
            continue

        class_id = category_mapping[cat_id]

        # COCO bbox: [x_min, y_min, width, height] (像素坐标)
        x_min, y_min, bbox_w, bbox_h = ann["bbox"]

        # 过滤无效框
        if bbox_w <= 0 or bbox_h <= 0:
            continue

        # 边界值裁剪
        x_min = max(0, min(x_min, img_width))
        y_min = max(0, min(y_min, img_height))
        bbox_w = min(bbox_w, img_width - x_min)
        bbox_h = min(bbox_h, img_height - y_min)

        # COCO [x_min, y_min, w, h] → YOLO [x_center, y_center, w, h]
        x_center = (x_min + bbox_w / 2.0) / img_width
        y_center = (y_min + bbox_h / 2.0) / img_height
        width = bbox_w / img_width
        height = bbox_h / img_height

        yolo_lines.append(
            f"{class_id} {x_center:.6f} {y_center:.6f} {width:.6f} {height:.6f}"
        )

```

```

# 保存 YOLO 标注文件
img_stem = Path(file_name).stem
txt_file = Path(output_dir) / f"{img_stem}.txt"
with open(txt_file, "w", encoding="utf-8") as f:
    f.write("\n".join(yolo_lines))

stats["converted"] += 1

logger.info(
    "COCO 转换完成: 总计 %d, 成功 %d, 跳过 %d",
    stats["total"],
    stats["converted"],
    stats["skipped"],
)
return stats

```

关键知识点讲解：

1. **COCO bbox 格式**：[x_min, y_min, width, height]，与 VOC 的 [xmin, ymin, xmax, ymax] 不同，这是初学者最常踩的坑
2. **category_mapping**：COCO 的 category_id 可能不是连续的（如 1, 3, 5），需要映射为 YOLO 的连续整数编号（0, 1, 2）
3. **自动编号**：当 class_mapping 为 None 时，按 categories 列表中的顺序自动从 0 编号，适合快速原型
4. **按 image_id 分组**：COCO 的所有标注在一个大数组中，需要先按 image_id 分组才能按图像输出

8.3 测试 COCO 转换

创建测试脚本：tools/convert_coco.py

```

#!/usr/bin/env python3
"""
COCO JSON → YOLO TXT 数据集格式转换脚本

功能：
    将 COCO 格式的 JSON 标注文件转换为 YOLO 格式的 TXT 标注文件
    支持完整的数据集处理流程：转换 + 划分 + yaml生成

COCO 格式说明：
    COCO JSON 文件包含三个主要部分：
    - images: 图像信息列表，包含 id, file_name, width, height
    - categories: 类别信息列表，包含 id, name
    - annotations: 标注信息列表，包含 image_id, category_id, bbox

COCO bbox 格式: [x_min, y_min, width, height] (像素坐标)
YOLO bbox 格式: [x_center, y_center, width, height] (归一化坐标)

使用方式：
    cd rsod-agent-platform/backend
    python ../tools/convert_coco.py

```

配置说明（修改下方配置区域）：

- COCO_JSON_FILE: COCO JSON 标注文件路径
- CLASS_MAPPING: 类别名称到数字ID的映射
- OUTPUT_DIR: YOLO 格式输出目录

输出目录结构：

```
yolo_dataset/
├── data.yaml
├── images/
│   ├── train/
│   ├── val/
│   └── test/
└── labels/
    ├── train/
    ├── val/
    └── test/
```

"""

```
import os
import random
import shutil
import sys

PROJECT_ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))

# — 配置区域（根据你的数据集修改） —————
# COCO JSON 标注文件路径
COCO_JSON_FILE = os.path.join(
    PROJECT_ROOT, "datasets/rsod/raw/annotations/instances_train.json"
)

# 原始图片目录（与 COCO JSON 中 file_name 对应的图片位置）
RAW_IMAGE_DIR = os.path.join(PROJECT_ROOT, "datasets/rsod/raw/images")

# YOLO 格式输出目录
OUTPUT_DIR = os.path.join(PROJECT_ROOT, "datasets/rsod/yolo_dataset")

# 类别映射（格式：{类别名称: 类别ID}）
# 如果设为 None, 则按 COCO categories 中的顺序自动编号（从 0 开始）
CLASS_MAPPING = {
    "aircraft": 0,
    "oiltank": 1,
    "overpass": 2,
    "playground": 3,
}

# 数据集划分比例
TRAIN_RATIO = 0.8
VAL_RATIO = 0.1
# TEST_RATIO = 0.1（剩余部分）

# 随机种子（保证划分结果可重复）
```



```
RANDOM_SEED = 42
```

```
# 支持的图片扩展名
```

```
IMAGE_EXTS = {".jpg", ".jpeg", ".png", ".bmp", ".tif", ".tiff"}
```

```
def coco_to_yolo(json_file: str, output_dir: str, class_mapping: dict = None) -> dict:
    """
```

```
    将 COCO JSON 格式转换为 YOLO TXT 格式
```

```
    参数:
```

```
        json_file: COCO JSON 标注文件路径
```

```
        output_dir: YOLO TXT 输出目录
```

```
        class_mapping: 类别映射字典 {类别名称: 类别ID}, 为 None 时自动编号
```

```
    返回:
```

```
        转换统计信息字典 {"total": 总数, "converted": 成功数, "skipped": 跳过数, "errors": 错误列表}
```

```
    转换流程:
```

```
        1. 读取 COCO JSON 文件
```

```
        2. 构建类别映射 (COCO category_id → YOLO class_id)
```

```
        3. 按 image_id 分组所有标注
```

```
        4. 遍历每张图像, 将 COCO bbox 转换为 YOLO 归一化坐标
```

```
        5. 每张图像输出一个 .txt 文件
```

```
    """
```

```
    import json
```

```
    from pathlib import Path
```

```
    os.makedirs(output_dir, exist_ok=True)
```

```
    stats = {"total": 0, "converted": 0, "skipped": 0, "errors": [], "image_files": []}
```

```
    # 读取 COCO JSON 文件
```

```
    try:
```

```
        with open(json_file, "r", encoding="utf-8") as f:
```

```
            coco_data = json.load(f)
```

```
    except (json.JSONDecodeError, FileNotFoundError) as e:
```

```
        print(f" [错误] COCO JSON 读取失败: {str(e)}")
```

```
        stats["errors"].append(str(e))
```

```
        return stats
```

```
    # 构建类别映射
```

```
    if class_mapping is None:
```

```
        # 自动按 categories 顺序编号 (从 0 开始)
```

```
        category_mapping = {
```

```
            cat["id"]: idx for idx, cat in enumerate(coco_data.get("categories", []))
```

```
        }
```

```
        print(f" 自动类别映射: {category_mapping}")
```

```
    else:
```

```
        # 使用用户提供的映射: 建立 coco_category_id → class_id 的关系
```

```
        cat_name_to_id = {
```

```
            cat["name"]: cat["id"] for cat in coco_data.get("categories", [])
```

```

    }
    category_mapping = {}
    for cat_name, class_id in class_mapping.items():
        if cat_name in cat_name_to_id:
            category_mapping[cat_name_to_id[cat_name]] = class_id
        else:
            print(f" [警告] COCO categories 中未找到类别 '{cat_name}'")

# 按 image_id 分组标注
image_annotations = {}
for ann in coco_data.get("annotations", []):
    img_id = ann["image_id"]
    if img_id not in image_annotations:
        image_annotations[img_id] = []
    image_annotations[img_id].append(ann)

# 遍历每张图像进行转换
images = coco_data.get("images", [])
stats["total"] = len(images)
print(f" COCO 转换开始: {len(images)} 张图像")

for img_info in images:
    img_id = img_info["id"]
    img_width = img_info["width"]
    img_height = img_info["height"]
    file_name = img_info["file_name"]

    # 检查图像尺寸有效性
    if img_width <= 0 or img_height <= 0:
        print(f" [警告] 图像 {file_name} 尺寸无效, 跳过")
        stats["skipped"] += 1
        continue

    # 收集图像文件名 (用于后续划分)
    stats["image_files"].append(file_name)

    # 转换标注
    yolo_lines = []
    for ann in image_annotations.get(img_id, []):
        # 检查类别是否在映射中
        cat_id = ann["category_id"]
        if cat_id not in category_mapping:
            continue

        class_id = category_mapping[cat_id]

        # COCO bbox: [x_min, y_min, width, height] (像素坐标)
        x_min, y_min, bbox_w, bbox_h = ann["bbox"]

        # 过滤无效框
        if bbox_w <= 0 or bbox_h <= 0:
            continue

```

```

# 边界值裁剪：确保坐标不超出图像范围
x_min = max(0, min(x_min, img_width))
y_min = max(0, min(y_min, img_height))
bbox_w = min(bbox_w, img_width - x_min)
bbox_h = min(bbox_h, img_height - y_min)

# COCO [x_min, y_min, w, h] → YOLO [x_center, y_center, w, h] (归一化)
x_center = (x_min + bbox_w / 2.0) / img_width
y_center = (y_min + bbox_h / 2.0) / img_height
width = bbox_w / img_width
height = bbox_h / img_height

yolo_lines.append(
    f"{class_id} {x_center:.6f} {y_center:.6f} {width:.6f} {height:.6f}"
)

# 保存 YOLO 标注文件
img_stem = Path(file_name).stem
txt_file = Path(output_dir) / f"{img_stem}.txt"
with open(txt_file, "w", encoding="utf-8") as f:
    f.write("\n".join(yolo_lines))

stats["converted"] += 1

print(
    f"  转换结果：总计 {stats['total']}, 成功 {stats['converted']}, 跳过 {stats['skipped']}"
)
return stats

def split_dataset(image_files, temp_label_dir):
    """
    按比例划分数据集并复制文件

    参数:
        image_files: 图像文件名列表
        temp_label_dir: 临时标注目录

    返回:
        None
    """
    # 设置随机种子，保证划分结果可重复
    random.seed(RANDOM_SEED)
    random.shuffle(image_files)

    # 计算划分索引
    total = len(image_files)
    train_end = int(total * TRAIN_RATIO)
    val_end = train_end + int(total * VAL_RATIO)

```

```
splits = {
    "train": image_files[:train_end],
    "val": image_files[train_end:val_end],
    "test": image_files[val_end:],
}
```

复制图片和标注到对应目录

```
for split_name, files in splits.items():
    img_out = os.path.join(OUTPUT_DIR, "images", split_name)
    lbl_out = os.path.join(OUTPUT_DIR, "labels", split_name)
    os.makedirs(img_out, exist_ok=True)
    os.makedirs(lbl_out, exist_ok=True)

    for filename in files:
        # 复制图片
        src_image = os.path.join(RAW_IMAGE_DIR, filename)
        if os.path.exists(src_image):
            dst_image = os.path.join(img_out, filename)
            shutil.copy2(src_image, dst_image)

        # 复制标注 (如果存在)
        basename = os.path.splitext(filename)[0]
        label_file = os.path.join(temp_label_dir, f"{basename}.txt")
        if os.path.exists(label_file):
            shutil.copy2(label_file, os.path.join(lbl_out, f"{basename}.txt"))
        else:
            open(os.path.join(lbl_out, f"{basename}.txt"), "w").close()

    print(f" {split_name}: {len(files)} 个")
```

```
def generate_yaml():
    """生成 YOLO 数据集配置文件"""
    # 获取类别列表
    if CLASS_MAPPING:
        class_names = sorted(CLASS_MAPPING.keys(), key=lambda x: CLASS_MAPPING[x])
    else:
        class_names = []

    # 构建 YAML 内容
    yaml_content = f"""path: {os.path.basename(OUTPUT_DIR)}
train: images/train
val: images/val
test: images/test
nc: {len(class_names)}
names:
"""

    for i, name in enumerate(class_names):
        yaml_content += f" {i}: {name}\n"

    # 写入 YAML 文件
    yaml_path = os.path.join(OUTPUT_DIR, "data.yaml")
```

```

with open(yaml_path, "w", encoding="utf-8") as f:
    f.write(yaml_content)
print(f"  配置文件已生成: {yaml_path}")

def main():
    """主函数: 执行完整的 COCO → YOLO 转换流程"""
    print("=" * 70)
    print("      COCO → YOLO 数据集转换流程")
    print("=" * 70)

    # 检查 COCO JSON 文件是否存在
    if not os.path.exists(COCO_JSON_FILE):
        print(f"\n[错误] COCO JSON 文件不存在: {COCO_JSON_FILE}")
        print("请检查配置区域中的 COCO_JSON_FILE 路径")
        sys.exit(1)

    # 检查原始图片目录是否存在
    if not os.path.exists(RAW_IMAGE_DIR):
        print(f"\n[错误] 原始图片目录不存在: {RAW_IMAGE_DIR}")
        print("请检查配置区域中的 RAW_IMAGE_DIR 路径")
        sys.exit(1)

    # — 步骤1: COCO转YOLO格式 —
    print("\n[1] COCO转YOLO格式")

    # 创建临时标注目录 (用于存放转换后的TXT文件)
    temp_label_dir = os.path.join(OUTPUT_DIR, "temp_labels")
    os.makedirs(temp_label_dir, exist_ok=True)

    # 执行转换
    stats = coco_to_yolo(COCO_JSON_FILE, temp_label_dir, CLASS_MAPPING)

    if stats["total"] == 0:
        print("\n[错误] 未找到任何图像数据, 请检查 COCO JSON 文件")
        shutil.rmtree(temp_label_dir, ignore_errors=True)
        sys.exit(1)

    # — 步骤2: 划分数据集 —
    print("\n[2] 划分数据集")
    split_dataset(stats["image_files"], temp_label_dir)

    # — 步骤3: 生成data.yaml —
    print("\n[3] 生成data.yaml")
    generate_yaml()

    # 清理临时目录
    shutil.rmtree(temp_label_dir, ignore_errors=True)

    # 输出完成信息
    print("\n" + "=" * 70)
    print(f"  处理完成! 输出目录: {OUTPUT_DIR}")

```

```
print("=" * 70)
```

```
if __name__ == "__main__":  
    main()
```

九、DataConverter 实现：LabelMe JSON → YOLO TXT

9.1 LabelMe JSON 结构解析

LabelMe JSON 结构解析

```
{  
  "version": "5.0.0",  
  "imagePath": "image001.jpg", - 图像文件名  
  "imageWidth": 800,           - 图像宽度 (像素)  
  "imageHeight": 600,          - 图像高度 (像素)  
  "shapes": [  
    {  
      "label": "airplane",      - 类别标签  
      "shape_type": "rectangle", - 形状类型  
      "points": [  
        [100, 80],             - 左上角坐标  
        [420, 320]             - 右下角坐标  
      ]  
    }, ...  
  ]  
}
```

注意：

- points 只有 2 个点时，分别是左上角和右下角
- points 有多个点时，需要取最小外接矩形
- shape_type 可能为 rectangle / polygon / circle 等

9.2 实现 LabelMe → YOLO 转换

文件：backend/app/training/data_converter.py (在 DataConverter 类中添加方法)

```
@staticmethod
```

```
def labelme_to_yolo(json_dir: str, output_dir: str, class_mapping: dict) -> dict:  
    """  
    LabelMe JSON 格式 → YOLO TXT 格式
```

转换逻辑：

1. 遍历 json_dir 下所有 .json 文件
2. 解析每个 JSON 获取图像尺寸和标注形状
3. 只处理 shape_type 为 "rectangle" 的标注

4. 从 points 中提取边界框, 转换为 YOLO 归一化坐标

Args:

json_dir: LabelMe JSON 文件目录
output_dir: YOLO TXT 输出目录
class_mapping: 类别映射 {类别名: 类别ID}

Returns:

转换统计信息字典

"""

```
os.makedirs(output_dir, exist_ok=True)
stats = {"total": 0, "converted": 0, "skipped": 0, "errors": []}

json_files = list(Path(json_dir).glob("*.json"))
if not json_files:
    logger.warning("LabelMe 转换: 目录 %s 下未找到 .json 文件", json_dir)
    return stats

logger.info(
    "LabelMe 转换开始: %d 个 JSON 文件 → %s", len(json_files), output_dir
)

for json_file in json_files:
    stats["total"] += 1
    try:
        with open(json_file, "r", encoding="utf-8") as f:
            labelme_data = json.load(f)

            img_width = labelme_data.get("imageWidth", 0)
            img_height = labelme_data.get("imageHeight", 0)

            if img_width <= 0 or img_height <= 0:
                logger.warning("JSON %s 图像尺寸无效, 跳过", json_file.name)
                stats["skipped"] += 1
                continue

            yolo_lines = []
            for shape in labelme_data.get("shapes", []):
                # 只处理矩形标注
                if shape.get("shape_type") != "rectangle":
                    continue

                class_name = shape.get("label", "").strip()
                if class_name not in class_mapping:
                    logger.warning(
                        "JSON %s 中类别 '%s' 不在映射表中, 跳过该目标",
                        json_file.name,
                        class_name,
                    )
                    continue

                class_id = class_mapping[class_name]
```

```

points = shape.get("points", [])

if len(points) < 2:
    continue

# 从 points 中提取最小和最大坐标
# 处理两种情况:
# 1. 只有 2 个点 (左上角和右下角)
# 2. 多个点 (取最小外接矩形)
all_x = [p[0] for p in points]
all_y = [p[1] for p in points]
xmin = min(all_x)
ymin = min(all_y)
xmax = max(all_x)
ymax = max(all_y)

# 边界值裁剪
xmin = max(0, min(xmin, img_width))
ymin = max(0, min(ymin, img_height))
xmax = max(0, min(xmax, img_width))
ymax = max(0, min(ymax, img_height))

# 过滤无效框
if xmax <= xmin or ymax <= ymin:
    continue

# 像素坐标 → YOLO 归一化坐标
x_center = (xmin + xmax) / 2.0 / img_width
y_center = (ymin + ymax) / 2.0 / img_height
width = (xmax - xmin) / img_width
height = (ymax - ymin) / img_height

yolo_lines.append(
    f"{class_id} {x_center:.6f} {y_center:.6f} "
    f"{width:.6f} {height:.6f}"
)

# 保存 YOLO 标注文件
txt_file = Path(output_dir) / f"{json_file.stem}.txt"
with open(txt_file, "w", encoding="utf-8") as f:
    f.write("\n".join(yolo_lines))

stats["converted"] += 1

except json.JSONDecodeError as e:
    logger.error("JSON 解析失败 %s: %s", json_file.name, str(e))
    stats["errors"].append(str(json_file.name))
    stats["skipped"] += 1
except Exception as e:
    logger.error("LabelMe 转换异常 %s: %s", json_file.name, str(e))
    stats["errors"].append(str(json_file.name))
    stats["skipped"] += 1

```



```

logger.info(
    "LabelMe 转换完成: 总计 %d, 成功 %d, 跳过 %d",
    stats["total"],
    stats["converted"],
    stats["skipped"],
)
return stats

```

关键知识点讲解:

1. **shape_type 过滤**: LabelMe 支持多种形状, 目标检测只需处理 "rectangle" 类型, 多边形 (polygon) 用于实例分割
2. **points 最小外接矩形**: 通过 `min()` / `max()` 提取所有点的边界框, 兼容 2 点和多点标注
3. **.get() 安全取值**: 使用 `.get("key", default)` 避免 `KeyError`, 增强代码鲁棒性

9.3 测试 LabelMe 转换

创建测试脚本: `tools/convert_labelme.py`

```

#!/usr/bin/env python3
"""
LabelMe JSON → YOLO TXT 数据集格式转换脚本

```

功能:

将 LabelMe 格式的 JSON 标注文件转换为 YOLO 格式的 TXT 标注文件
支持完整的数据集处理流程: 转换 + 划分 + yaml生成

LabelMe 格式说明:

LabelMe 标注工具生成的 JSON 文件包含:

- imagePath: 图像文件路径
- imageWidth: 图像宽度
- imageHeight: 图像高度
- shapes: 标注形状列表, 每个形状包含:
 - label: 类别名称
 - shape_type: 形状类型 (rectangle/polygon/circle等)
 - points: 坐标点列表

本脚本只处理 shape_type 为 "rectangle" 的标注

LabelMe rectangle points: `[[x1, y1], [x2, y2]]` (两个对角点, 像素坐标)

YOLO bbox 格式: `[x_center, y_center, width, height]` (归一化坐标)

使用方式:

```

cd rsod-agent-platform/backend
python ../tools/convert_labelme.py

```

配置说明 (修改下方配置区域):

- LABELME_JSON_DIR: LabelMe JSON 文件所在目录
- CLASS_MAPPING: 类别名称到数字ID的映射
- OUTPUT_DIR: YOLO 格式输出目录

输出目录结构:

```
yolo_dataset/
├── data.yaml
├── images/
│   ├── train/
│   ├── val/
│   └── test/
└── labels/
    ├── train/
    ├── val/
    └── test/
```

"""

```
import os
import sys
import shutil
import random

PROJECT_ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))

# — 配置区域 (根据你的数据集修改) —————
# LabelMe JSON 文件所在目录
LABELME_JSON_DIR = os.path.join(
    PROJECT_ROOT, "datasets/rsod/raw/labelme_annotations"
)

# 原始图片目录 (与 LabelMe JSON 中 imagePath 对应的图片位置)
# 如果 imagePath 是相对路径, 图片应放在此目录下
RAW_IMAGE_DIR = os.path.join(PROJECT_ROOT, "datasets/rsod/raw/images")

# YOLO 格式输出目录
OUTPUT_DIR = os.path.join(PROJECT_ROOT, "datasets/rsod/yolo_dataset")

# 类别映射 (格式: {类别名称: 类别ID})
CLASS_MAPPING = {
    "aircraft": 0,
    "oiltank": 1,
    "overpass": 2,
    "playground": 3,
}

# 数据集划分比例
TRAIN_RATIO = 0.8
VAL_RATIO = 0.1
# TEST_RATIO = 0.1 (剩余部分)

# 随机种子 (保证划分结果可重复)
RANDOM_SEED = 42

# 支持的图片扩展名
IMAGE_EXTS = {".jpg", ".jpeg", ".png", ".bmp", ".tif", ".tiff"}
```

```
def labelme_to_yolo(json_dir: str, output_dir: str, class_mapping: dict) -> dict:
    """
```

将 LabelMe JSON 格式转换为 YOLO TXT 格式

参数:

json_dir: LabelMe JSON 文件所在目录
output_dir: YOLO TXT 输出目录
class_mapping: 类别映射字典 {类别名称: 类别ID}

返回:

转换统计信息字典 {"total": 总数, "converted": 成功数, "skipped": 跳过数, "errors": 错误列表, "image_files": 图像文件列表}

转换流程:

1. 遍历 json_dir 下所有 .json 文件
2. 解析每个 JSON 获取图像尺寸和标注形状
3. 只处理 shape_type 为 "rectangle" 的标注
4. 从 points 中提取边界框 (取最小外接矩形)
5. 转换为 YOLO 归一化坐标
6. 每张图像输出一个 .txt 文件

```
"""
```

```
import json
```

```
from pathlib import Path
```

```
os.makedirs(output_dir, exist_ok=True)
```

```
stats = {"total": 0, "converted": 0, "skipped": 0, "errors": [], "image_files": []}
```

```
# 获取所有 JSON 文件
```

```
json_files = list(Path(json_dir).glob("*.json"))
```

```
if not json_files:
```

```
    print(f" [警告] 目录 {json_dir} 下未找到 .json 文件")
```

```
    return stats
```

```
print(f" LabelMe 转换开始: {len(json_files)} 个 JSON 文件")
```

```
# 遍历每个 JSON 文件
```

```
for json_file in json_files:
```

```
    stats["total"] += 1
```

```
    try:
```

```
        # 读取 JSON 文件
```

```
        with open(json_file, "r", encoding="utf-8") as f:
```

```
            labelme_data = json.load(f)
```

```
        # 获取图像尺寸
```

```
        img_width = labelme_data.get("imageWidth", 0)
```

```
        img_height = labelme_data.get("imageHeight", 0)
```

```
        # 检查图像尺寸有效性
```

```
        if img_width <= 0 or img_height <= 0:
```

```
            print(f" [警告] JSON {json_file.name} 图像尺寸无效, 跳过")
```

```

        stats["skipped"] += 1
        continue

# 获取图像文件名
image_path = labelme_data.get("imagePath", "")
image_filename = os.path.basename(image_path)
stats["image_files"].append(image_filename)

# 转换标注
yolo_lines = []
for shape in labelme_data.get("shapes", []):
    # 只处理矩形标注 (其他形状如 polygon 不支持)
    if shape.get("shape_type") != "rectangle":
        continue

    # 获取类别名称
    class_name = shape.get("label", "").strip()
    if class_name not in class_mapping:
        print(f" [警告] JSON {json_file.name} 中类别 '{class_name}' 不在映射表
中, 跳过")
        continue

    class_id = class_mapping[class_name]
    points = shape.get("points", [])

    # 检查点数量
    if len(points) < 2:
        continue

    # 从 points 中提取最小和最大坐标
    # 处理两种情况:
    # 1. 只有 2 个点 (左上角和右下角)
    # 2. 多个点 (取最小外接矩形)
    all_x = [p[0] for p in points]
    all_y = [p[1] for p in points]
    xmin = min(all_x)
    ymin = min(all_y)
    xmax = max(all_x)
    ymax = max(all_y)

    # 边界值裁剪: 确保坐标不超出图像范围
    xmin = max(0, min(xmin, img_width))
    ymin = max(0, min(ymin, img_height))
    xmax = max(0, min(xmax, img_width))
    ymax = max(0, min(ymax, img_height))

    # 过滤无效框 (宽或高为 0)
    if xmax <= xmin or ymax <= ymin:
        continue

    # 像素坐标 → YOLO 归一化坐标
    x_center = (xmin + xmax) / 2.0 / img_width

```

```

        y_center = (ymin + ymax) / 2.0 / img_height
        width = (xmax - xmin) / img_width
        height = (ymax - ymin) / img_height

        yolo_lines.append(
            f"{class_id} {x_center:.6f} {y_center:.6f} {width:.6f} {height:.6f}"
        )

    # 保存 YOLO 标注文件
    txt_file = Path(output_dir) / f"{json_file.stem}.txt"
    with open(txt_file, "w", encoding="utf-8") as f:
        f.write("\n".join(yolo_lines))

    stats["converted"] += 1

except json.JSONDecodeError as e:
    print(f" [错误] JSON 解析失败 {json_file.name}: {str(e)}")
    stats["errors"].append(str(json_file.name))
    stats["skipped"] += 1
except Exception as e:
    print(f" [错误] LabelMe 转换异常 {json_file.name}: {str(e)}")
    stats["errors"].append(str(json_file.name))
    stats["skipped"] += 1

print(f" 转换结果: 总计 {stats['total']}, 成功 {stats['converted']}, 跳过 {stats['skipped']}")
return stats

def split_dataset(image_files, temp_label_dir):
    """
    按比例划分数据集并复制文件

    参数:
        image_files: 图像文件名列表
        temp_label_dir: 临时标注目录

    返回:
        None
    """
    # 设置随机种子, 保证划分结果可重复
    random.seed(RANDOM_SEED)
    random.shuffle(image_files)

    # 计算划分索引
    total = len(image_files)
    train_end = int(total * TRAIN_RATIO)
    val_end = train_end + int(total * VAL_RATIO)

    splits = {
        "train": image_files[:train_end],
        "val": image_files[train_end:val_end],
    }

```

```

        "test": image_files[val_end:],
    }

# 复制图片和标注到对应目录
for split_name, files in splits.items():
    img_out = os.path.join(OUTPUT_DIR, "images", split_name)
    lbl_out = os.path.join(OUTPUT_DIR, "labels", split_name)
    os.makedirs(img_out, exist_ok=True)
    os.makedirs(lbl_out, exist_ok=True)

    for filename in files:
        # 复制图片
        src_image = os.path.join(RAW_IMAGE_DIR, filename)
        if os.path.exists(src_image):
            dst_image = os.path.join(img_out, filename)
            shutil.copy2(src_image, dst_image)

            # 复制标注 (如果存在)
            basename = os.path.splitext(filename)[0]
            label_file = os.path.join(temp_label_dir, f"{basename}.txt")
            if os.path.exists(label_file):
                shutil.copy2(label_file, os.path.join(lbl_out, f"{basename}.txt"))
            else:
                open(os.path.join(lbl_out, f"{basename}.txt"), "w").close()
        else:
            # 图片不存在, 尝试从 JSON 目录查找
            for ext in IMAGE_EXTS:
                alt_image_path = os.path.join(LABELME_JSON_DIR, basename + ext)
                if os.path.exists(alt_image_path):
                    shutil.copy2(alt_image_path, os.path.join(img_out, filename))
                    label_file = os.path.join(temp_label_dir, f"{basename}.txt")
                    if os.path.exists(label_file):
                        shutil.copy2(label_file, os.path.join(lbl_out, f"
{basename}.txt"))
                    else:
                        open(os.path.join(lbl_out, f"{basename}.txt"), "w").close()
                    break

            print(f" {split_name}: {len(files)} ↑")

def generate_yaml():
    """生成 YOLO 数据集配置文件"""
    # 获取类别列表
    class_names = sorted(CLASS_MAPPING.keys(), key=lambda x: CLASS_MAPPING[x])

    # 构建 YAML 内容
    yaml_content = f"""path: ./{os.path.basename(OUTPUT_DIR)}
train: images/train
val: images/val
test: images/test
nc: {len(class_names)}

```

```

names:
"""
    for i, name in enumerate(class_names):
        yaml_content += f"    {i}: {name}\n"

# 写入 YAML 文件
yaml_path = os.path.join(OUTPUT_DIR, "data.yaml")
with open(yaml_path, "w", encoding="utf-8") as f:
    f.write(yaml_content)
print(f"    配置文件已生成: {yaml_path}")

def main():
    """主函数: 执行完整的 LabelMe → YOLO 转换流程"""
    print("=" * 70)
    print("        LabelMe → YOLO 数据集转换流程")
    print("=" * 70)

    # 检查 LabelMe JSON 目录是否存在
    if not os.path.exists(LABELME_JSON_DIR):
        print(f"\n[错误] LabelMe JSON 目录不存在: {LABELME_JSON_DIR}")
        print("请检查配置区域中的 LABELME_JSON_DIR 路径")
        sys.exit(1)

    # 检查原始图片目录是否存在
    if not os.path.exists(RAW_IMAGE_DIR):
        print(f"\n[错误] 原始图片目录不存在: {RAW_IMAGE_DIR}")
        print("请检查配置区域中的 RAW_IMAGE_DIR 路径")
        sys.exit(1)

    # — 步骤1: LabelMe转YOLO格式 —
    print("\n[1] LabelMe转YOLO格式")

    # 创建临时标注目录 (用于存放转换后的TXT文件)
    temp_label_dir = os.path.join(OUTPUT_DIR, "temp_labels")
    os.makedirs(temp_label_dir, exist_ok=True)

    # 执行转换
    stats = labelme_to_yolo(LABELME_JSON_DIR, temp_label_dir, CLASS_MAPPING)

    if stats["total"] == 0:
        print("\n[错误] 未找到任何 JSON 文件, 请检查 LABELME_JSON_DIR 目录")
        shutil.rmtree(temp_label_dir, ignore_errors=True)
        sys.exit(1)

    # — 步骤2: 划分数据集 —
    print("\n[2] 划分数据集")
    split_dataset(stats["image_files"], temp_label_dir)

    # — 步骤3: 生成data.yaml —
    print("\n[3] 生成data.yaml")
    generate_yaml()

```

```

# 清理临时目录
shutil.rmtree(temp_label_dir, ignore_errors=True)

# 输出完成信息
print("\n" + "=" * 70)
print(f"  处理完成! 输出目录: {OUTPUT_DIR}")
print("=" * 70)

if __name__ == "__main__":
    main()

```

十、数据集目录结构组织

10.1 YOLO 标准目录结构

YOLOv11（通过 Ultralytics 库）要求数据集按以下结构组织：

```

datasets/{scene_name}/
├── images/
│   ├── train/                # 训练集图像
│   │   ├── img001.jpg
│   │   ├── img002.jpg
│   │   └── ...
│   ├── val/                  # 验证集图像
│   │   ├── img101.jpg
│   │   └── ...
│   └── test/                  # 测试集图像（可选）
│       ├── img201.jpg
│       └── ...
├── labels/
│   ├── train/                # 训练集标注（YOLO TXT 格式）
│   │   ├── img001.txt        # 文件名必须与图像一致
│   │   ├── img002.txt
│   │   └── ...
│   ├── val/                  # 验证集标注
│   │   ├── img101.txt
│   │   └── ...
│   └── test/                  # 测试集标注（可选）
│       ├── img201.txt
│       └── ...
└── data.yaml                  # 数据集配置文件

```

10.2 关键规则

YOLO 数据集目录规则

1. images/ 和 labels/ 是平级目录，不是嵌套关系
2. train/val/test 是 images/ 和 labels/ 的子目录
3. 图像文件名和标注文件名必须完全一致（仅扩展名不同）
images/train/img001.jpg ↔ labels/train/img001.txt
4. 每张图像必须有对应的标注文件（即使标注为空文件）
5. data.yaml 位于数据集根目录

10.3 目录组织脚本

文件：backend/app/training/dataset_splitter.py

```
"""
```

数据集划分与目录组织工具

职责：

- 将图像和标注文件按指定比例划分到 train/val/test 目录
- 自动创建目录结构
- 验证图像与标注的配对完整性

使用方式：

```
from app.training.dataset_splitter import DatasetSplitter
```

```
splitter = DatasetSplitter()
```

```
splitter.organize_dataset(
```

```
    image_dir="datasets/rsod/yolo_dataset/images",
```

```
    label_dir="datasets/rsod/yolo_dataset/labels",
```

```
    output_dir="datasets/rsod",
```

```
    train_ratio=0.8,
```

```
    val_ratio=0.1,
```

```
    test_ratio=0.1
```

```
)
```

```
"""
```

```
import os
```

```
import random
```

```
import shutil
```

```
from pathlib import Path
```

```
from app.core.logger import get_logger
```

```
logger = get_logger(__name__)
```

```
class DatasetSplitter:
```

```
    """数据集划分与目录组织工具"""
```

```
    @staticmethod
```

```

def organize_dataset(
    image_dir: str,
    label_dir: str,
    output_dir: str,
    train_ratio: float = 0.8,
    val_ratio: float = 0.1,
    test_ratio: float = 0.1,
    seed: int = 42,
) -> dict:
    """
    将图像和标注文件按比例划分到 train/val/test 目录

    Args:
        image_dir: 原始图像目录（所有图像在同一目录）
        label_dir: 原始标注目录（所有标注在同一目录）
        output_dir: 输出目录（将创建标准 YOLO 目录结构）
        train_ratio: 训练集比例
        val_ratio: 验证集比例
        test_ratio: 测试集比例
        seed: 随机种子（确保可重复）

    Returns:
        划分统计信息
    """
    random.seed(seed)

    # 获取所有图像文件
    image_extensions = {".jpg", ".jpeg", ".png", ".bmp", ".tif", ".tiff"}
    image_files = sorted(
        [
            f
            for f in Path(image_dir).iterdir()
            if f.suffix.lower() in image_extensions
        ]
    )

    if not image_files:
        logger.error("图像目录 %s 下未找到图像文件", image_dir)
        return {"error": "未找到图像文件"}

    logger.info("找到 %d 张图像, 开始划分...", len(image_files))

    # 随机打乱
    random.shuffle(image_files)

    # 计算划分边界
    total = len(image_files)
    train_end = int(total * train_ratio)
    val_end = train_end + int(total * val_ratio)

    splits = {
        "train": image_files[:train_end],

```

```

        "val": image_files[train_end:val_end],
        "test": image_files[val_end:],
    }

# 创建目录结构并复制文件
stats = {"train": 0, "val": 0, "test": 0, "missing_labels": []}

for split_name, files in splits.items():
    # 创建目录
    img_out = Path(output_dir) / "images" / split_name
    lbl_out = Path(output_dir) / "labels" / split_name
    img_out.mkdir(parents=True, exist_ok=True)
    lbl_out.mkdir(parents=True, exist_ok=True)

    for img_file in files:
        # 复制图像
        shutil.copy2(img_file, img_out / img_file.name)

        # 查找并复制对应的标注文件
        label_file = Path(label_dir) / f"{img_file.stem}.txt"
        if label_file.exists():
            shutil.copy2(label_file, lbl_out / label_file.name)
            stats[split_name] += 1
        else:
            # 没有标注文件时创建空文件
            empty_label = lbl_out / f"{img_file.stem}.txt"
            empty_label.touch()
            stats["missing_labels"].append(img_file.name)
            stats[split_name] += 1
            logger.warning(
                "图像 %s 无对应标注文件, 已创建空标注", img_file.name
            )

logger.info(
    "数据集划分完成: train=%d, val=%d, test=%d, 缺失标注=%d",
    stats["train"],
    stats["val"],
    stats["test"],
    len(stats["missing_labels"]),
)
return stats

@staticmethod
def generate_data_yaml(
    output_dir: str,
    class_names: list,
    class_names_cn: list = None,
) -> str:
    """
    自动生成 data.yaml 配置文件

    Args:

```

```

        output_dir: 数据集根目录
        class_names: 类别名称列表 (英文, 按 class_id 顺序)
        class_names_cn: 类别中文名称列表 (可选, 用于前端显示)

Returns:
    生成的 data.yaml 文件路径
"""
import yaml

data_config = {
    "path": f"./{os.path.basename(output_dir)}",
    "train": "images/train",
    "val": "images/val",
    "test": "images/test",
    "nc": len(class_names),
    "names": {i: name for i, name in enumerate(class_names)},
}

if class_names_cn:
    data_config["names_cn"] = {i: name for i, name in enumerate(class_names_cn)}

yaml_path = Path(output_dir) / "data.yaml"
with open(yaml_path, "w", encoding="utf-8") as f:
    yaml.dump(
        data_config,
        f,
        default_flow_style=False,
        allow_unicode=True,
        sort_keys=False,
    )

logger.info("data.yaml 已生成: %s", yaml_path)
return str(yaml_path)

```

关键知识点讲解:

1. `random.seed(seed)`: 设置随机种子确保每次划分结果一致, 便于实验可重复
2. `shutil.copy2`: 复制文件并保留元数据 (如修改时间), 比 `shutil.copy` 更可靠
3. 空标注文件: YOLO 允许没有目标的图像, 对应的 `.txt` 文件存在但内容为空即可
4. 比例计算: 先计算 `train_end` 和 `val_end` 边界, 然后用切片划分, 避免浮点精度问题

10.4 使用示例

```

cd rsod-agent-platform
python -c "
import sys
sys.path.insert(0, 'backend')
from app.training.dataset_splitter import DatasetSplitter

splitter = DatasetSplitter()

```

```
stats = splitter.organize_dataset(
    image_dir='datasets/rsod/yolo_dataset/images',
    label_dir='datasets/rsod/yolo_dataset/labels',
    output_dir='datasets/rsod',
    train_ratio=0.8,
    val_ratio=0.1,
    test_ratio=0.1,
)
print(f'训练集: {stats["train"]} 张')
print(f'验证集: {stats["val"]} 张')
print(f'测试集: {stats["test"]} 张')
"
```

十一、data.yaml 配置文件生成

11.1 data.yaml 作用

`data.yaml` 是 YOLOV11 训练时的数据集配置文件，告诉模型：

- 数据在哪里 (train/val/test 路径)
- 有多少个类别 (nc)
- 每个类别的名称 (names)

11.2 data.yaml 格式

```
# datasets/rsod/yolo_dataset/data.yaml

# 数据集根路径（可以是绝对路径或相对于训练脚本的相对路径）
path: ./yolo_dataset

# 各子集路径（相对于 path）
train: images/train
val: images/val
test: images/test

# 类别数量
nc: 4

# 类别名称（索引必须从 0 开始，与标注文件中的 class_id 对应）
names:
  0: aircraft
  1: oiltank
  2: overpass
  3: playground
```

11.3 各场景 data.yaml 示例

智慧农业：

```
path: ../datasets/agriculture
train: images/train
val: images/val
test: images/test
```

```
nc: 4
names:
  0: healthy_leaf
  1: powdery_mildew
  2: leaf_spot
  3: rust
```

工业检测:

```
path: ../datasets/industrial
train: images/train
val: images/val
test: images/test
```

```
nc: 6
names:
  0: crazing
  1: inclusion
  2: patches
  3: pitted_surface
  4: rolled-in_scale
  5: scratches
```

遥感检测:

```
path: ../datasets/rsod
train: images/train
val: images/val
test: images/test
```

```
nc: 4
names:
  0: aircraft
  1: oiltank
  2: overpass
  3: playground
```

医疗影像:

```

path: ../datasets/medical
train: images/train
val: images/val
test: images/test

nc: 3
names:
  0: normal
  1: benign_tumor
  2: malignant_tumor

```

交通/安防:

```

path: ../datasets/traffic
train: images/train
val: images/val
test: images/test

nc: 5
names:
  0: car
  1: bus
  2: truck
  3: pedestrian
  4: cyclist

```

11.4 自动生成 data.yaml 的工具函数

在 `dataset_splitter.py` 中添加:

```

@staticmethod
def generate_data_yaml(
    output_dir: str,
    class_names: list,
    class_names_cn: list = None,
) -> str:
    """
    自动生成 data.yaml 配置文件

    Args:
        output_dir: 数据集根目录
        class_names: 类别名称列表 (英文, 按 class_id 顺序)
        class_names_cn: 类别中文名称列表 (可选, 用于前端显示)

    Returns:
        生成的 data.yaml 文件路径
    """
    import yaml

    data_config = {

```

```

        "path": f"./{os.path.basename(output_dir)}",
        "train": "images/train",
        "val": "images/val",
        "test": "images/test",
        "nc": len(class_names),
        "names": {i: name for i, name in enumerate(class_names)},
    }

    if class_names_cn:
        data_config["names_cn"] = {i: name for i, name in enumerate(class_names_cn)}

    yaml_path = Path(output_dir) / "data.yaml"
    with open(yaml_path, "w", encoding="utf-8") as f:
        yaml.dump(
            data_config,
            f,
            default_flow_style=False,
            allow_unicode=True,
            sort_keys=False,
        )

    logger.info("data.yaml 已生成: %s", yaml_path)
    return str(yaml_path)

```

11.5 使用示例

```

cd rsod-agent-platform
python -c "
import sys
sys.path.insert(0, 'backend')
from app.training.dataset_splitter import DatasetSplitter

# 生成遥感场景的 data.yaml
DatasetSplitter.generate_data_yaml(
    output_dir='datasets/rsod/yolo_dataset',
    class_names=['aircraft', 'oiltank', 'overpass', 'playground'],
    class_names_cn=['飞机', '油罐', '立交桥', '操场'],
)
print('data.yaml 已生成! ')
"

```

十二、数据集划分策略

12.1 划分比例建议

数据集划分策略

标准划分（推荐）：

Train 80% | Val 10% | Test 10%

小数据集（< 500 张）：

Train 70% | Val 15% | Test 15%

大数据集（> 5000 张）：

Train 90% | Val 5% | Test 5%

说明：

- Train — 用于模型训练，数据越多越好
- Val — 用于训练过程中评估，调超参数
- Test — 用于最终评估，训练期间不参与

12.2 划分注意事项

1. **类别均衡**：确保 train/val/test 中各类别的比例大致一致，避免某个类别只在训练集中出现
2. **随机打乱**：划分前先随机打乱图像顺序，避免按文件名排序导致的偏差
3. **固定种子**：使用固定 `seed` 确保每次划分结果一致，便于实验对比
4. **图像-标注配对**：图像和标注必须一起划分，不能分开打乱

十三、格式转换完整 workflow

13.1 工作流总览

数据集准备完整工作流

Step 1: 获取原始数据

- Roboflow 下载（已是 YOLO 格式 → 跳到 Step 4）
- Kaggle 下载（检查标注格式）
- 官方数据集下载（VOC/COCO/自定义格式）
- 自有图像 + LabelImg/LabelMe 标注

Step 2: 格式转换

- VOC XML → `DataConverter.voc_to_yolo()`
- COCO JSON → `DataConverter.coco_to_yolo()`
- LabelMe → `DataConverter.labelme_to_yolo()`

Step 3: 数据集划分

- `DatasetSplitter.organize_dataset()`

→ 按 8:1:1 划分到 train/val/test

Step 4: 目录整理

└─ 确保符合 YOLO 标准目录结构
 images/train, images/val, images/test
 labels/train, labels/val, labels/test

Step 5: 生成 data.yaml

└─ DatasetSplitter.generate_data_yaml()

Step 6: 验证数据集

└─ tools/verify_dataset.py (检查完整性)

13.2 数据集验证脚本

文件: `tools/verify_dataset.py`

```
#!/usr/bin/env python3
```

```
"""
```

YOLO 数据集验证脚本（增强版）

功能:

对转换后的 YOLO 格式数据集进行全面验证，确保数据质量满足训练要求

验证内容:

1. 目录结构是否符合 YOLO 规范 (images/train、images/val、labels/train、labels/val)
2. 图像与标注文件是否一一对应 (文件名匹配检查)
3. 标注文件格式是否正确 (每行必须为 5 个值: class_id x_center y_center width height)
4. 类别 ID 是否有效 (必须为整数)
5. 归一化坐标是否在 [0, 1] 范围内 (YOLO格式要求)
6. 统计各类别的标注数量和占比 (生成分布直方图)
7. 类别不平衡检测和警告 (比例 > 10:1 视为严重不平衡)
8. 边界框统计 (平均尺寸、最小/最大尺寸、小目标占比)
9. 每个 split (train/val/test) 的详细统计

使用方式:

```
cd rsod-agent-platform/backend  
python ../tools/verify_dataset.py
```

或指定数据集目录:

```
python ../tools/verify_dataset.py /path/to/dataset
```

输出:

详细的验证报告，包含统计信息、警告和建议

数据集格式要求:

```
yolo_dataset/  
└─ data.yaml          # 数据集配置文件 (必须包含 names 字段)  
└─ images/  
    └─ train/         # 训练集图片
```

```

├── val/           # 验证集图片
├── test/          # 测试集图片 (可选)
└── labels/
    ├── train/     # 训练集标注 (.txt)
    ├── val/       # 验证集标注 (.txt)
    └── test/      # 测试集标注 (.txt)

```

YOLO 标注格式:

每行一个目标, 格式为:

```
class_id x_center y_center width height
```

其中坐标为归一化值 (0~1)

```
"""
```

```
import os
import sys
from pathlib import Path
```

支持的图片扩展名集合

```
IMAGE_EXTS = {".jpg", ".jpeg", ".png", ".bmp", ".tif", ".tiff"}
```

```
def load_yaml_classes(dataset_dir: Path) -> dict:
```

```
"""
```

加载 data.yaml 中的类别定义 (纯文本解析, 不依赖 yaml 库)

参数:

dataset_dir: 数据集目录路径 (Path 对象)

返回:

类别 ID 到类别名称的映射字典, 格式 {0: "aircraft", 1: "oiltank", ...}

解析逻辑:

1. 查找 data.yaml 文件
2. 逐行读取, 找到 "names:" 字段开始的部分
3. 解析缩进的类别定义 (格式: 数字ID: 类别名称)
4. 返回解析结果

```
"""
```

```
yaml_path = dataset_dir / "data.yaml"
```

```
if not yaml_path.exists():
```

```
    return {}
```

```
try:
```

```
    names = {}
```

```
    with open(yaml_path, "r", encoding="utf-8") as f:
```

```
        in_names = False # 是否进入 names 字段区域
```

```
        for line in f:
```

```
            line = line.strip()
```

```
            # 找到 names: 字段, 开始解析
```

```
            if line.startswith("names:"):

```

```
                in_names = True

```

```
                continue

```

```
            # 在 names 区域内, 解析类别定义

```

```

        if in_names and line:
            # 检查是否为数字开头的类别定义
            if line[0].isdigit():
                parts = line.split(":", 1)
                if len(parts) == 2:
                    class_id = int(parts[0].strip())
                    class_name = parts[1].strip()
                    names[class_id] = class_name
            # 遇到空行, 结束 names 解析
            elif in_names and not line:
                break
    return names
except Exception:
    return {}

```

```
def verify_dataset(dataset_dir: str) -> dict:
```

```
"""
```

验证 YOLO 数据集完整性, 返回详细的验证结果

参数:

dataset_dir: 数据集目录路径 (字符串)

返回:

验证结果字典, 包含以下字段:

```

    total_images: 图像总数
    total_labels: 标注文件总数
    total_annotations: 标注目标总数
    missing_labels: 缺少标注的图像列表
    missing_images: 缺少图像的标注列表
    empty_labels: 空标注文件数量
    invalid_format: 格式错误列表
    out_of_range: 坐标越界列表
    class_distribution: 类别分布字典
    class_names: 类别名称映射
    bbox_stats: 边界框统计信息
    split_stats: 各 split 的统计信息
    has_warnings: 是否存在警告

```

```
"""
```

初始化验证结果字典

```

results = {
    "total_images": 0, # 图像总数
    "total_labels": 0, # 标注文件总数
    "total_annotations": 0, # 标注目标总数
    "missing_labels": [], # 缺少标注的图像列表
    "missing_images": [], # 缺少图像的标注列表
    "empty_labels": 0, # 空标注文件数量
    "invalid_format": [], # 格式错误列表
    "out_of_range": [], # 坐标越界列表
    "class_distribution": {}, # 类别分布 {class_id: count}
    "class_names": {}, # 类别名称映射
    "bbox_stats": { # 边界框统计

```

```

        "total": 0, # 边界框总数
        "avg_width": 0, # 平均宽度 (累加和, 最后求平均)
        "avg_height": 0, # 平均高度 (累加和, 最后求平均)
        "max_width": 0, # 最大宽度
        "max_height": 0, # 最大高度
        "min_width": float("inf"), # 最小宽度 (初始化为无穷大)
        "min_height": float("inf"), # 最小高度 (初始化为无穷大)
        "small_boxes": 0, # 小目标数量 (面积 < 0.001)
        "large_boxes": 0, # 大目标数量 (面积 > 0.5)
    },
    "split_stats": {}, # 各 split 的统计信息
    "has_warnings": False, # 是否存在警告
}

# 将字符串路径转换为 Path 对象
dataset_path = Path(dataset_dir)
# 加载类别名称映射
class_names = load_yaml_classes(dataset_path)
results["class_names"] = class_names

# 遍历每个 split (train/val/test)
for split in ["train", "val", "test"]:
    img_dir = dataset_path / "images" / split # 图片目录
    lbl_dir = dataset_path / "labels" / split # 标注目录

    # 初始化当前 split 的统计结果
    split_result = {
        "images": 0, # 图片数量
        "labels": 0, # 标注数量
        "annotations": 0, # 标注目标数量
        "missing_labels": 0, # 缺少标注的图片数
        "missing_images": 0, # 缺少图片的标注数
        "class_distribution": {}, # 类别分布
    }

    # 检查图片目录是否存在 (test 目录可选)
    if not img_dir.exists():
        if split != "test":
            print(f"[警告] 缺少目录: {img_dir}")
        results["split_stats"][split] = split_result
        continue

    # 检查标注目录是否存在
    if not lbl_dir.exists():
        print(f"[警告] 缺少目录: {lbl_dir}")
        results["split_stats"][split] = split_result
        continue

    # 获取所有图片和标注文件的文件名 (不含扩展名)
    image_files = {
        f.stem for f in img_dir.iterdir() if f.suffix.lower() in IMAGE_EXTS
    }

```

```

label_files = {f.stem for f in lbl_dir.iterdir() if f.suffix == ".txt"}

# 找出不匹配的文件
missing_labels = image_files - label_files # 有图片无标注
missing_images = label_files - image_files # 有标注无图片

# 更新统计信息
split_result["images"] = len(image_files)
split_result["labels"] = len(label_files)
split_result["missing_labels"] = len(missing_labels)
split_result["missing_images"] = len(missing_images)

# 更新全局统计信息
results["missing_labels"].extend([f"{split}/{name}" for name in missing_labels])
results["missing_images"].extend([f"{split}/{name}" for name in missing_images])
results["total_images"] += len(image_files)
results["total_labels"] += len(label_files)

# 存储边界框尺寸信息 (用于后续统计)
bbox_widths = []
bbox_heights = []

# 遍历所有标注文件, 验证格式和内容
for label_file in lbl_dir.glob("*.txt"):
    content = label_file.read_text(encoding="utf-8").strip()

    # 检查标注文件是否为空
    if not content:
        results["empty_labels"] += 1
        continue

    # 逐行解析标注内容
    for line_num, line in enumerate(content.split("\n"), 1):
        parts = line.strip().split()

        # 检查每行是否包含 5 个值
        if len(parts) != 5:
            results["invalid_format"].append(
                f"{split}/{label_file.name}:{line_num} (期望 5 个值, 实际 {len(parts)})"
            )
            continue

        # 检查 class_id 是否为整数
        try:
            class_id = int(parts[0])
        except ValueError:
            results["invalid_format"].append(
                f"{split}/{label_file.name}:{line_num} (class_id 非整数)"
            )
            continue

```

```

# 更新类别分布统计
results["class_distribution"][class_id] = (
    results["class_distribution"].get(class_id, 0) + 1
)
split_result["class_distribution"][class_id] = (
    split_result["class_distribution"].get(class_id, 0) + 1
)
# 更新标注目标总数
results["total_annotations"] += 1
split_result["annotations"] += 1

# 检查坐标值是否为浮点数且在有效范围
try:
    coords = [float(v) for v in parts[1:]]
    x_center, y_center, width, height = coords

    # 检查归一化坐标是否在 [0, 1] 范围内
    for i, v in enumerate(coords):
        if v < 0 or v > 1:
            field_names = ["x_center", "y_center", "width", "height"]
            results["out_of_range"].append(
                f"{split}/{label_file.name}:{line_num} {field_names[i]}={
{v:.6f}}"
            )
            break

    # 收集边界框尺寸信息
    bbox_widths.append(width)
    bbox_heights.append(height)

except ValueError:
    results["invalid_format"].append(
        f"{split}/{label_file.name}:{line_num} (坐标值非浮点数)"
    )

# 更新边界框统计信息
if bbox_widths:
    results["bbox_stats"]["total"] += len(bbox_widths)
    results["bbox_stats"]["avg_width"] += sum(bbox_widths)
    results["bbox_stats"]["avg_height"] += sum(bbox_heights)
    results["bbox_stats"]["max_width"] = max(
        results["bbox_stats"]["max_width"], max(bbox_widths)
    )
    results["bbox_stats"]["max_height"] = max(
        results["bbox_stats"]["max_height"], max(bbox_heights)
    )
    results["bbox_stats"]["min_width"] = min(
        results["bbox_stats"]["min_width"], min(bbox_widths)
    )
    results["bbox_stats"]["min_height"] = min(
        results["bbox_stats"]["min_height"], min(bbox_heights)
    )

```

```

        # 统计小目标 (面积 < 0.001) 和大目标 (面积 > 0.5)
        results["bbox_stats"]["small_boxes"] += sum(
            1 for w, h in zip(bbox_widths, bbox_heights) if w * h < 0.001
        )
        results["bbox_stats"]["large_boxes"] += sum(
            1 for w, h in zip(bbox_widths, bbox_heights) if w * h > 0.5
        )

    # 保存当前 split 的统计结果
    results["split_stats"][split] = split_result

# 计算平均宽度和高度 (除以总数)
if results["bbox_stats"]["total"] > 0:
    results["bbox_stats"]["avg_width"] /= results["bbox_stats"]["total"]
    results["bbox_stats"]["avg_height"] /= results["bbox_stats"]["total"]

return results


def print_report(results: dict):
    """
    打印格式化的验证报告

    参数:
        results: verify_dataset() 返回的验证结果字典

    输出格式:
        - 总体统计 (图像数、标注数、目标数)
        - Split 统计 (train/val/test 各数据集的详细信息)
        - 警告信息 (缺少标注、格式错误、坐标越界等)
        - 类别分布 (柱状图可视化)
        - 边界框统计 (尺寸分布、小目标占比)
        - 最终结论 (通过/存在问题)
    """
    print("\n" + "=" * 70)
    print("                YOLO 数据集验证报告")
    print("=" * 70)

    # — 总体统计 —
    print("\n [总体统计]")
    print(f"    图像总数: {results['total_images']}")
    print(f"    标注文件数: {results['total_labels']}")
    print(f"    标注目标数: {results['total_annotations']}")
    print(f"    空标注文件: {results['empty_labels']}")
    print(
        f"    平均每图标注: {results['total_annotations'] / results['total_images']:.2f}"
        if results["total_images"] > 0
        else "    平均每图标注: N/A"
    )

    # — Split 统计 —
    print("\n [Split 统计]")

```



```

for split in ["train", "val", "test"]:
    stats = results["split_stats"].get(split, {})
    print(
        f"    {split}: {stats.get('images', 0)} 图像, {stats.get('labels', 0)} 标注,
{stats.get('annotations', 0)} 目标"
    )

# — 缺少标注文件警告 —
if results["missing_labels"]:
    print(f"\n [警告] 缺少标注文件 ({len(results['missing_labels'])} 个): ")
    for name in results["missing_labels"][:5]:
        print(f"    - {name}")
    if len(results["missing_labels"]) > 5:
        print(f"    ... 还有 {len(results['missing_labels']) - 5} 个")
    results["has_warnings"] = True

# — 缺少图像文件警告 —
if results["missing_images"]:
    print(f"\n [警告] 缺少图像文件 ({len(results['missing_images'])} 个): ")
    for name in results["missing_images"][:5]:
        print(f"    - {name}")
    if len(results["missing_images"]) > 5:
        print(f"    ... 还有 {len(results['missing_images']) - 5} 个")
    results["has_warnings"] = True

# — 格式错误警告 —
if results["invalid_format"]:
    print(f"\n [错误] 格式错误 ({len(results['invalid_format'])} 处): ")
    for item in results["invalid_format"][:5]:
        print(f"    - {item}")
    if len(results["invalid_format"]) > 5:
        print(f"    ... 还有 {len(results['invalid_format']) - 5} 处")
    results["has_warnings"] = True

# — 坐标越界警告 —
if results["out_of_range"]:
    print(f"\n [警告] 坐标越界 ({len(results['out_of_range'])} 处): ")
    for item in results["out_of_range"][:5]:
        print(f"    - {item}")
    if len(results["out_of_range"]) > 5:
        print(f"    ... 还有 {len(results['out_of_range']) - 5} 处")
    results["has_warnings"] = True

# — 类别分布统计 —
print("\n [类别分布]")
class_names = results["class_names"]
total = results["total_annotations"] or 1

if results["class_distribution"]:
    # 计算类别不平衡比例
    max_count = max(results["class_distribution"].values())
    min_count = min(results["class_distribution"].values())

```

```

imbalance_ratio = max_count / min_count if min_count > 0 else float("inf")

# 打印类别分布 (带柱状图)
for class_id in sorted(results["class_distribution"].keys()):
    count = results["class_distribution"][class_id]
    percentage = (count / total) * 100
    class_name = class_names.get(class_id, f"class_{class_id}")
    # 计算柱状图长度 (最大40个字符)
    bar_length = int((count / max_count) * 40) if max_count > 0 else 0
    bar = "█" * bar_length + "░" * (40 - bar_length)
    print(
        f"      {class_id:2d}. {class_name:12s} {count:6d} ↑ ({percentage:5.2f}%)
{bar}"
    )

# 类别不平衡警告
if imbalance_ratio > 10:
    print(
        f"\n  [警告] 类别严重不平衡! 最大/最小类别比例 = {imbalance_ratio:.1f}:1"
    )
    print("      建议: 增加少数类样本或使用数据增强技术")
    results["has_warnings"] = True
elif imbalance_ratio > 5:
    print(f"\n  [提示] 类别存在一定不平衡, 比例 = {imbalance_ratio:.1f}:1")

# — 边界框统计 —
print("\n  [边界框统计]")
bbox = results["bbox_stats"]
if bbox["total"] > 0:
    print(f"      边界框总数: {bbox['total']}")
    print(f"      平均宽度: {bbox['avg_width']:.4f}")
    print(f"      平均高度: {bbox['avg_height']:.4f}")
    print(f"      最小宽度: {bbox['min_width']:.4f}")
    print(f"      最小高度: {bbox['min_height']:.4f}")
    print(f"      最大宽度: {bbox['max_width']:.4f}")
    print(f"      最大高度: {bbox['max_height']:.4f}")
    print(
        f"      小目标 (面积<0.001): {bbox['small_boxes']} ↑ ({bbox['small_boxes'] /
bbox['total'] * 100:.1f}%)
    )
    print(
        f"      大目标 (面积>0.5): {bbox['large_boxes']} ↑ ({bbox['large_boxes'] /
bbox['total'] * 100:.1f}%)
    )

# 小目标占比提示
if bbox["small_boxes"] / bbox["total"] > 0.3:
    print("\n  [提示] 小目标占比较高 (>30%), 建议使用合适的锚框配置")

# — 最终结论 —
print(f"\n{'-' * 70}")
if results["has_warnings"]:

```

```

        print("  结果: ⚠ 数据集存在问题或警告, 请根据上述信息修复")
    else:
        print("  结果: ✓ 数据集验证通过, 可以开始训练")
    print(f"{'-' * 70}\n")

if __name__ == "__main__":
    """主函数: 执行数据集验证流程"""
    # 计算项目根目录 (tools/verify_dataset.py → rsod-agent-platform/)
    PROJECT_ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
    # 默认数据集目录
    DATASET_DIR = os.path.join(PROJECT_ROOT, "datasets/rsod/yolo_dataset")

    # 支持通过命令行参数指定数据集目录
    if len(sys.argv) > 1:
        DATASET_DIR = sys.argv[1]

    # 检查数据集目录是否存在
    if not os.path.exists(DATASET_DIR):
        print(f"[错误] 数据集目录不存在: {DATASET_DIR}")
        sys.exit(1)

    # 执行验证并打印报告
    results = verify_dataset(DATASET_DIR)
    print_report(results)

```

13.3 运行验证

```

cd rsod-agent-platform/backend

# 验证数据集 (指定场景目录)
python ../tools/verify_dataset.py

```

预期输出:

```

=====
                        YOLO 数据集验证报告
=====

[总体统计]
  图像总数: 936
  标注文件数: 936
  标注目标数: 7400
  空标注文件: 0
  平均每图标注: 7.91

[Split 统计]
  train: 748 图像, 748 标注, 5787 目标
  val: 93 图像, 93 标注, 807 目标
  test: 95 图像, 95 标注, 806 目标

```

[类别分布]

0. aircraft	5374 个 (72.62%)	
1. oiltank	1698 个 (22.95%)	
2. overpass	178 个 (2.41%)	
3. playground	150 个 (2.03%)	

[警告] 类别严重不平衡! 最大/最小类别比例 = 35.8:1

建议: 增加少数类样本或使用数据增强技术

[边界框统计]

边界框总数: 7400

平均宽度: 0.0799

平均高度: 0.0886

最小宽度: 0.0081

最小高度: 0.0109

最大宽度: 0.7181

最大高度: 0.8862

小目标 (面积<0.001): 915 个 (12.4%)

大目标 (面积>0.5): 2 个 (0.0%)

结果: ⚠ 数据集存在问题或警告, 请根据上述信息修复

13.4 标注文件越界修复

如遇验证报告提示标注文件越界边界框, 可以通过下方工具类修复:

```
#tools/fix_bbox.py
```

```
"""
```

修复 YOLO 标注文件中的越界边界框

功能:

1. 遍历所有标注文件, 检查坐标是否在 [0, 1] 范围内
2. 将越界坐标裁剪到有效范围
3. 删除无效边界框 (宽或高 <= 0)
4. 输出修复统计报告

使用方式:

```
cd rsod-agent-platform/backend
```

```
python ../tools/fix_bbox.py
```

```
"""
```

```
import os
```

```
PROJECT_ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
```

```
DATASET_DIR = os.path.join(PROJECT_ROOT, "datasets/rsod/yolo_dataset")
```

```

def fix_bbox_coordinates(label_dir):
    """
    修复单个目录下的标注文件

    参数:
        label_dir: 标注文件目录

    返回:
        修复统计字典 {"fixed": 修复数, "deleted": 删除数, "total": 总数}
    """
    stats = {"fixed": 0, "deleted": 0, "total": 0, "files_affected": 0}

    if not os.path.exists(label_dir):
        return stats

    for filename in os.listdir(label_dir):
        if not filename.endswith(".txt"):
            continue

        filepath = os.path.join(label_dir, filename)
        with open(filepath, "r", encoding="utf-8") as f:
            lines = f.readlines()

        new_lines = []
        file_fixed = 0
        file_deleted = 0

        for line in lines:
            line = line.strip()
            if not line:
                continue

            parts = line.split()
            if len(parts) != 5:
                file_deleted += 1
                continue

            try:
                class_id = int(parts[0])
                x_center = float(parts[1])
                y_center = float(parts[2])
                width = float(parts[3])
                height = float(parts[4])
            except ValueError:
                file_deleted += 1
                continue

            # 裁剪越界坐标到 [0, 1]
            x_center = max(0.0, min(1.0, x_center))
            y_center = max(0.0, min(1.0, y_center))
            width = max(0.0, min(1.0, width))
            height = max(0.0, min(1.0, height))

```

```

# 检查是否修复了越界
if (
    float(parts[1]) < 0
    or float(parts[1]) > 1
    or float(parts[2]) < 0
    or float(parts[2]) > 1
    or float(parts[3]) < 0
    or float(parts[3]) > 1
    or float(parts[4]) < 0
    or float(parts[4]) > 1
):
    file_fixed += 1

# 过滤无效框 (宽或高 <= 0)
if width <= 0 or height <= 0:
    file_deleted += 1
    continue

new_lines.append(
    f"{class_id} {x_center:.6f} {y_center:.6f} {width:.6f} {height:.6f}"
)

# 只有在变化时写入
if file_fixed > 0 or file_deleted > 0 or len(new_lines) != len(lines):
    with open(filepath, "w", encoding="utf-8") as f:
        f.write("\n".join(new_lines))

stats["fixed"] += file_fixed
stats["deleted"] += file_deleted
stats["total"] += len(lines)
if file_fixed > 0 or file_deleted > 0:
    stats["files_affected"] += 1

return stats

def main():
    """主函数: 修复所有 split 的标注文件"""
    print("=" * 70)
    print("      修复 YOLO 标注文件中的越界边界框")
    print("=" * 70)

    total_stats = {"fixed": 0, "deleted": 0, "total": 0, "files_affected": 0}

    # 遍历每个 split
    for split in ["train", "val", "test"]:
        label_dir = os.path.join(DATASET_DIR, "labels", split)
        print(f"\n[处理 {split}]")

        if not os.path.exists(label_dir):
            print("  目录不存在, 跳过")

```

```

        continue

    stats = fix_bbox_coordinates(label_dir)
    print(f"  修复: {stats['fixed']} 个边界框")
    print(f"  删除: {stats['deleted']} 个无效边界框")
    print(f"  影响文件: {stats['files_affected']} 个")

    # 累加统计
    for key in total_stats:
        total_stats[key] += stats[key]

    print("\n" + "=" * 70)
    print("  修复完成!")
    print(f"  总计边界框: {total_stats['total']}")
    print(f"  修复越界: {total_stats['fixed']} 个")
    print(f"  删除无效: {total_stats['deleted']} 个")
    print(f"  影响文件: {total_stats['files_affected']} 个")
    print("=" * 70)

if __name__ == "__main__":
    main()

```

也可以在数据转换之前，先进行VOC XML 标注文件中的越界坐标修复

```

# tools/fix_xml_coords.py
"""
修复 VOC XML 标注文件中的越界坐标

功能：
1. 遍历所有 XML 文件，检查边界框坐标是否越界
2. 将负数坐标和超出图像范围的坐标裁剪到有效范围
3. 删除无效边界框（宽或高 <= 0）
4. 输出修复统计报告

使用方式：
cd rsod-agent-platform/backend
python ../tools/fix_xml_coords.py

修复规则：
- xmin < 0 → xmin = 0
- ymin < 0 → ymin = 0
- xmax > width → xmax = width
- ymax > height → ymax = height
- xmax <= xmin 或 ymax <= ymin → 删除该目标
"""

import os
import xml.etree.ElementTree as ET

PROJECT_ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))

```

```
ANNOTATION_DIR = os.path.join(PROJECT_ROOT, "datasets/rsod/raw/annotations")
```

```
def fix_xml_file(xml_path):  
    """  
    修复单个 XML 文件中的越界坐标  
  
    参数:  
        xml_path: XML 文件路径  
  
    返回:  
        修复统计字典 {"fixed": 修复数, "deleted": 删除数}  
    """  
    stats = {"fixed": 0, "deleted": 0, "total": 0}  
  
    try:  
        tree = ET.parse(xml_path)  
        root = tree.getroot()  
  
        # 获取图片尺寸  
        size = root.find("size")  
        if size is None:  
            return stats  
  
        width_elem = size.find("width")  
        height_elem = size.find("height")  
        if width_elem is None or height_elem is None:  
            return stats  
  
        if width_elem.text is None or height_elem.text is None:  
            return stats  
  
        img_width = int(width_elem.text)  
        img_height = int(height_elem.text)  
  
        if img_width <= 0 or img_height <= 0:  
            return stats  
  
        # 遍历所有目标对象  
        objects = root.findall("object")  
        stats["total"] = len(objects)  
  
        # 收集需要删除的目标  
        to_delete = []  
  
        for obj in objects:  
            bbox = obj.find("bndbox")  
            if bbox is None:  
                continue  
  
            xmin_elem = bbox.find("xmin")  
            ymin_elem = bbox.find("ymin")
```



```

xmax_elem = bbox.find("xmax")
ymax_elem = bbox.find("ymax")

if (
    xmin_elem is None
    or ymin_elem is None
    or xmax_elem is None
    or ymax_elem is None
):
    continue

if (
    xmin_elem.text is None
    or ymin_elem.text is None
    or xmax_elem.text is None
    or ymax_elem.text is None
):
    continue

# 获取原始坐标
xmin = float(xmin_elem.text)
ymin = float(ymin_elem.text)
xmax = float(xmax_elem.text)
ymax = float(ymax_elem.text)

# 记录是否有变化
changed = False

# 边界值裁剪
if xmin < 0:
    xmin = 0
    changed = True
if ymin < 0:
    ymin = 0
    changed = True
if xmax > img_width:
    xmax = img_width
    changed = True
if ymax > img_height:
    ymax = img_height
    changed = True

# 检查边界框有效性
if xmax <= xmin or ymax <= ymin:
    to_delete.append(obj)
    stats["deleted"] += 1
    continue

# 如果有变化, 更新 XML
if changed:
    xmin_elem.text = str(int(xmin))
    ymin_elem.text = str(int(ymin))

```

```

        xmax_elem.text = str(int(xmax))
        ymax_elem.text = str(int(ymax))
        stats["fixed"] += 1

# 删除无效目标
for obj in to_delete:
    root.remove(obj)

# 只有在变化时写入
if stats["fixed"] > 0 or stats["deleted"] > 0:
    tree.write(xml_path, encoding="utf-8", xml_declaration=True)

except Exception as e:
    print(f" [错误] 处理 {os.path.basename(xml_path)}: {str(e)}")

return stats

def main():
    """主函数: 修复所有 XML 文件"""
    print("=" * 70)
    print("      修复 VOC XML 标注文件中的越界坐标")
    print("=" * 70)

    total_stats = {"fixed": 0, "deleted": 0, "total": 0, "files_affected": 0}

# 获取所有 XML 文件
xml_files = [f for f in os.listdir(ANNOTATION_DIR) if f.lower().endswith(".xml")]

print(f"\n找到 {len(xml_files)} 个 XML 文件")
print("正在处理...")

# 遍历处理每个 XML 文件
for xml_file in xml_files:
    xml_path = os.path.join(ANNOTATION_DIR, xml_file)
    stats = fix_xml_file(xml_path)

    if stats["fixed"] > 0 or stats["deleted"] > 0:
        total_stats["files_affected"] += 1
        print(f" {xml_file}: 修复 {stats['fixed']} 个, 删除 {stats['deleted']} 个")

    total_stats["fixed"] += stats["fixed"]
    total_stats["deleted"] += stats["deleted"]
    total_stats["total"] += stats["total"]

print("\n" + "=" * 70)
print("  修复完成! ")
print(f"  总计目标数: {total_stats['total']}")
print(f"  修复越界: {total_stats['fixed']} 个")
print(f"  删除无效: {total_stats['deleted']} 个")
print(f"  影响文件: {total_stats['files_affected']} 个")
print("=" * 70)

```

```
if __name__ == "__main__":  
    main()
```

十四、各组场景数据集准备指南

14.1 推荐操作流程

各组数据集准备 Step-by-Step

Step 1: 确定场景方向

- 与组员讨论，选择一个具体场景
- 确定 3~6 个检测类别
- 类别命名用英文小写+连字符（如 surface-defect）

Step 2: 搜索并获取数据集

- 首选 Roboflow（可直接导出 YOLO 格式）
- 备选 Kaggle（需检查标注格式）
- 至少准备 200 张以上图像（建议 500+）

Step 3: 检查标注格式

- 已是 YOLO 格式 → 跳到 Step 5
- VOC XML → 使用 `DataConverter.voc_to_yolo()`
- COCO JSON → 使用 `DataConverter.coco_to_yolo()`
- LabelMe JSON → 使用 `DataConverter.labelme_to_yolo()`

Step 4: 如需自行标注

- 安装 LabelImg: `pip install labelImg`
- 标注 50~100 张图像
- 使用 `DataConverter` 转换为 YOLO 格式

Step 5: 组织目录结构

- 使用 `DatasetSplitter.organize_dataset()`
- 或手动创建 train/val/test 目录

Step 6: 生成 data.yaml

- 使用 `DatasetSplitter.generate_data_yaml()`

Step 7: 验证数据集

- `python tools/verify_dataset.py datasets/{your_scene}`

14.2 数据量建议

场景	最低数据量	推荐数据量	说明
智慧农业	200 张	500~2000 张	叶片图像变化较小
工业检测	300 张	1000~5000 张	缺陷形态多变
遥感检测	500 张	1000~5000 张	目标尺寸变化大
医疗影像	200 张	500~2000 张	标注需专业背景
交通/安防	500 张	2000~10000 张	场景复杂度高

14.3 数据质量检查清单

- ☐ 图像清晰度足够，目标可辨认
- ☐ 标注框紧密包围目标，不留过多空白
- ☐ 类别命名统一（避免 `car` 和 `Car` 混用）
- ☐ 各类别数据量大致均衡（避免某类别只有几张图）
- ☐ 无重复图像
- ☐ 无损坏/无法打开的图像

十五、常见问题排查

Q1：Roboflow 下载的数据集目录结构与 YOLO 标准不一致

原因：Roboflow 导出时将 `images` 和 `labels` 放在同一级目录（如 `train/images` 和 `train/labels`），而 YOLO 标准是 `images/train` 和 `labels/train`。

解决：

```
# 方法 1：使用 DatasetSplitter 重组
# 方法 2：手动重组
cd datasets/rsod

# 创建标准目录
mkdir -p images/train images/val images/test
mkdir -p labels/train labels/val labels/test

# 移动文件
mv train/images/* images/train/ 2>/dev/null
mv train/labels/* labels/train/ 2>/dev/null
mv valid/images/* images/val/ 2>/dev/null
mv valid/labels/* labels/val/ 2>/dev/null
mv test/images/* images/test/ 2>/dev/null
mv test/labels/* labels/test/ 2>/dev/null

# 清理旧目录
rm -rf train valid test
```

Q2: COCO 转换后所有标注的 class_id 都是 0

原因: COCO 的 `category_id` 从 1 开始 (1, 2, 3...)，自动编号时从 0 开始映射。如果只提供了一个类别映射且映射值为 0，所有标注都会变成 0。

解决: 检查 `class_mapping` 参数是否正确覆盖了所有需要转换的类别。如果使用自动编号，确认 COCO JSON 中的 `categories` 列表顺序正确。

Q3: VOC XML 解析报错 `xml.etree.ElementTree.ParseError`

原因: XML 文件损坏或包含非法字符。

解决:

```
# 在 DataConverter 中已有异常处理，会跳过损坏的文件
# 如需修复，可以用文本编辑器打开 XML 检查
# 常见原因：文件名包含特殊字符、编码声明错误
```

Q4: 标注文件中坐标值超出 [0, 1] 范围

原因: 标注工具的框画出了图像边界，或图像尺寸信息不正确。

解决: `DataConverter` 中已实现边界值裁剪 (`max(0, min(x, img_width))`)，转换后的坐标不会超出范围。如果仍有问题，检查原始标注中的图像尺寸是否正确。

Q5: 训练时报错 `FileNotFoundError: No images found`

原因: `data.yaml` 中的路径配置不正确。

解决:

```
# 确保 path 是数据集根目录的绝对路径或相对于训练脚本的正确相对路径
# 推荐在训练时使用绝对路径:
path: /Users/lily/Desktop/rsod-agent-platform/datasets/rsod
train: images/train
val: images/val
```

Q6: `pip install labelImg` 报错

原因: `LabelImg` 依赖 `PyQt5`，在某些系统上安装复杂。

解决:

```
# macOS
pip install labelImg

# 如果 PyQt5 安装失败, 尝试:
brew install pyqt5
pip install labelImg

# 或使用 LabelMe 替代 (无 PyQt 依赖问题):
pip install labelme
```

Q7: 数据集划分后 train 和 val 的类别分布不均匀

原因: 简单随机划分未考虑类别均衡。

解决: 对于类别严重不均衡的数据集, 使用分层采样。修改 `DatasetSplitter.organize_dataset()` 中的划分逻辑, 按类别分组后再按比例分配。对于 Day 5 的阶段, 数据量足够大时 (>500 张), 简单随机划分通常已足够。

Q8: 图像扩展名不一致 (.JPG vs .jpg)

原因: 不同来源的图像文件扩展名大小写不一致。

解决:

```
# 批量转换为小写扩展名
cd datasets/rsod/raw/images
for f in *.JPG; do
    mv "$f" "${f%.JPG}.jpg"
done
```

Day 5 验收自查清单

逐项确认以下全部通过:

```
# == 数据准备验证 ==

# 1. 检查数据集目录结构
ls -R datasets/{your_scene}/
# 预期: 看到 images/train, images/val, labels/train, labels/val

# 2. 检查图像数量
find datasets/{your_scene}/images/train -type f | wc -l
find datasets/{your_scene}/images/val -type f | wc -l
# 预期: train 至少 100+, val 至少 20+

# 3. 检查标注文件数量与图像一致
diff <(ls datasets/{your_scene}/images/train/ | sed 's/\.([^.]*$//') \
    <(ls datasets/{your_scene}/labels/train/ | sed 's/\.([^.]*$//')
# 预期: 无输出 (文件名完全一致)
```

```
# 4. 检查标注文件格式
head -5 datasets/{your_scene}/labels/train/*.txt | head -20
# 预期: 每行 5 个值, 第一个是整数, 后四个是 [0,1] 范围内的浮点数

# 5. 检查 data.yaml
cat datasets/{your_scene}/data.yaml
# 预期: 包含 path, train, val, nc, names 字段

# 6. 运行数据集验证脚本
python tools/verify_dataset.py datasets/{your_scene}
# 预期: 验证通过

# == 代码验证 ==

# 7. DataConverter 可正确导入
cd backend
source .venv/bin/activate
python -c "from app.training.data_converter import DataConverter; print('DataConverter 导入成功')"
```

```
# 8. DatasetSplitter 可正确导入
python -c "from app.training.dataset_splitter import DatasetSplitter;
print('DatasetSplitter 导入成功')"
```

全部通过后, **Day 5 数据集获取 + 格式转换完成**, 为 Day 6 的数据验证与模型训练做好准备。

文档版本: v1.0

适用项目: 基于 YOLOv11 的目标检测智能体平台

最后更新: 2026-07-03